Tugas Besar 1 IF3270 Pembelajaran Mesin

Feedforward Neural Network

Disusun oleh Kelompok 55:

Sebastian Enrico Nathanael (13523134)

Jonathan Kenan Budianto (13523139)

Mahesa Fadhillah Andre (13523140)

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

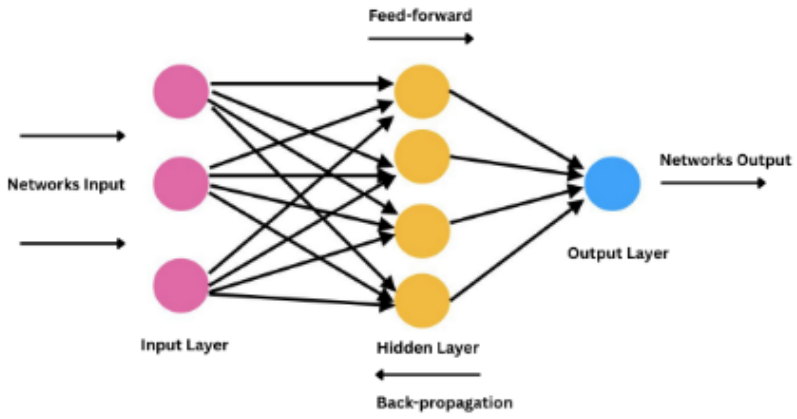
DAFTAR ISI

DAFTAR ISI **2**
BAB I : DESKRIPSI PEROSALAN **3**
1.1 Latar Belakang 3
1.2 Dataset 4
1.3 Tujuan 5
BAB II : PEMBAHASAN **6**
2.1 Penjelasan Implementasi 6
2.1.1 Deskripsi kelas beserta deskripsi atribut dan methodnya 6
2.1.2 Penjelasan forward propagation 18
2.1.2.1 Alur Komputasi 18
2.1.2.2 Implementasi melalui Autodiff 18
2.1.2.3 Peran RMSNorm dalam Forward Pass 19
2.1.3 Penjelasan backward propagation dan weight update 19
2.1.3.1 Automatic Differentiation (Autodiff) 19
2.1.3.2 Proses Backward Pass 19
2.1.3.3 Chain Rule pada Graf Komputasi 20
2.1.3.3 Chain Rule pada Graf Komputasi 20
2.1.3.4 Weight Update 20
2.1.3.5 Training Loop 21
2.2 Hasil Pengujian 21
2.2.1 Preprocessing Dataset 21
2.2.2 Pengaruh depth dan width 23
2.2.2.1 Eksperimen Variasi Lebar (Width) dengan Kedalaman Tetap 23
2.2.2.2 Eksperimen Variasi Kedalaman (Depth) dengan Lebar Tetap 25
2.2.3 Pengaruh fungsi aktivasi 27
2.2.4 Pengaruh learning rate 29
2.2.5 Pengaruh inisialisasi bobot 30
2.2.6 Pengaruh regularisasi 31
2.2.7 Pengaruh normalisasi RMSNorm 33
2.2.8 Perbandingan dengan library sklearn 35
BAB III : KESIMPULAN DAN SARAN **37**
3.1 Kesimpulan 37
3.2 Saran 38
BAB IV : PEMBAGIAN TUGAS TIAP ANGGOTA KELOMPOK **39**
BAB V : REFERENSI **40**
BAB VI : LAMPIRAN **41**
6.1 Lampiran Surat Pernyataan Penggunaan AI 41
6.2 Lampiran Gambar Hasil Notebook 45

BAB I : DESKRIPSI PEROSALAN

1.1 Latar Belakang

Perkembangan teknologi kecerdasan buatan, khususnya deep learning, telah membawa perubahan revolusioner dalam memecahkan berbagai masalah pemodelan data yang kompleks, mulai dari pengenalan pola, visi komputer, hingga klasifikasi data tabular. Arsitektur paling mendasar yang menjadi tulang punggung dari berbagai model deep learning tingkat lanjut saat ini adalah Feedforward Neural Network (FFNN). Meskipun saat ini telah tersedia berbagai framework tingkat tinggi seperti PyTorch dan TensorFlow yang sangat memudahkan praktisi dalam mengembangkan model, ketergantungan penuh pada framework tersebut seringkali mengaburkan pemahaman fundamental mengenai komputasi matematis di dalam jaringan, atau yang sering dikenal dengan fenomena "kotak hitam" (black box).



Berangkat dari urgensi tersebut, Tugas Besar 1 IF3270 Pembelajaran Mesin mengharuskan implementasi *Feedforward Neural Network* (FFNN) murni dari nol (*from scratch*) menggunakan bahasa pemrograman Python. Dalam pengerjaannya, implementasi ini sangat dibatasi dan hanya diperbolehkan memanfaatkan library untuk keperluan komputasi matematika dasar seperti NumPy, tanpa menggunakan *framework deep learning* yang sudah ada. Pembatasan ini secara khusus dirancang untuk melatih kemampuan menerjemahkan konsep-konsep aljabar linear dan kalkulus multivariabel ke dalam barisan kode komputasi yang efisien.

Melalui pengerjaan model dari awal, proses pembelajaran algoritma menjadi jauh lebih transparan. Tantangan dalam membangun *neural network* murni menuntut pemahaman yang solid terkait proses komputasi yang terjadi di tiap lapisan neuron, mulai dari penerusan sinyal komputasi melalui transformasi linear dan aktivasi non-linear (*forward propagation*), perhitungan gradien kesalahan yang menerapkan prinsip aturan rantai atau *chain rule* (*backpropagation*), hingga proses optimisasi bobot menggunakan algoritma *gradient descent*. Dengan demikian, fondasi konseptual arsitektur *machine learning* dapat terbangun dengan kuat sebagai landasan sebelum bereksperimen pada penyelesaian data di dunia nyata.

1.2 Dataset

Dataset yang digunakan dalam tugas ini adalah Global Student Placement & Salary Dataset yang terdiri dari 10.000 data mahasiswa dari berbagai negara di seluruh dunia. Dataset ini berisi informasi mengenai performa akademik mahasiswa serta *outcome* penempatan kerja setelah lulus. Setiap entri memiliki sebelas atribut, yaitu *cgpa* (nilai IPK skala 4.0–10.0), *backlogs* (jumlah mata kuliah yang belum lulus), *college_tier* (tingkatan institusi), *country* (negara kelulusan), *university_ranking_band* (kategori peringkat global universitas), *internship_count* (jumlah internship yang diselesaikan), *aptitude_score* (skor kemampuan kuantitatif), *communication_score* (skor kemampuan komunikasi), *specialization* (bidang akademik), *industry* (industri tempat bekerja), dan *internship_quality_score* (skor kualitas pengalaman internship). Kolom yang menjadi target prediksi adalah placement_status, yang nilainya adalah *Placed* atau *Not Placed*.

Column	**Description**	**Type**
cgpa	Cumulative Grade Point Average (skala 4.0–10.0)	Float
backlogs	Jumlah mata kuliah yang belum lulus	Integer
college_tier	Tingkatan institusi	Categorical
country	Negara kelulusan	Categorical
university_ranking_band	Kategori peringkat global universitas	Categorical
internship_count	Jumlah internship yang telah diselesaikan	Integer
aptitude_score	Skor tes kemampuan kuantitatif	Integer
communication_score	Skor kemampuan komunikasi	Integer
specialization	Bidang akademik	Categorical
industry	Industri tempat bekerja	Categorical
internship_quality_score	Skor kualitas pengalaman internship	Float
placement_status	Status penempatan kerja	Categorical

1.3 Tujuan

Berdasarkan latar belakang yang telah dipaparkan, pengerjaan Tugas Besar ini dirancang untuk mencapai sejumlah sasaran teknis dan pemahaman konseptual terkait pengembangan model machine learning dari tingkat dasar. Adapun tujuan-tujuan utama yang ingin dicapai melalui implementasi kode dan eksperimen ini adalah sebagai berikut:

1. Membangun arsitektur Feedforward Neural Network (FFNN) secara mandiri

   Mengimplementasikan model FFNN from scratch dengan hanya menggunakan bahasa pemrograman Python dan library komputasi numerik NumPy.

2. Mengimplementasikan dan memahami algoritma komputasi inti

   Menerapkan proses forward propagation serta komputasi gradien secara otomatis melalui computation graph menggunakan metode Automatic Differentiation (Autograd).

3. Mengembangkan variasi komponen arsitektur jaringan

   Mengimplementasikan berbagai kelengkapan model, termasuk fungsi aktivasi standar dan tambahan (seperti ELU dan Leaky ReLU), fungsi loss, metode inisialisasi bobot tingkat lanjut (Xavier dan He), optimisator gradient descent dan Adam, hingga normalisasi RMSNorm.

4. Menganalisis pengaruh hyperparameter terhadap kinerja model

   Melakukan eksperimen komprehensif untuk mengamati metrik evaluasi model dengan memvariasikan kedalaman jaringan (depth), lebar jaringan (width), tingkat pembelajaran (learning rate), dan fungsi aktivasi.

5. Melakukan validasi kinerja melalui studi komparatif

   Membandingkan performa arsitektur FFNN buatan sendiri dengan implementasi dari library scikit-learn (MLPClassifier) sebagai tolok ukur (baseline) validasi dalam memprediksi data pada dataset Global Student Placement & Salary.

BAB II : PEMBAHASAN

2.1 Penjelasan Implementasi

2.1.1 Deskripsi kelas beserta deskripsi atribut dan methodnya

2.1.1.1 Kelas Tensor (src/engine/autodiff.py)

Kelas Tensor adalah struktur data inti yang membungkus array dari NumPy. Kelas ini berfungsi untuk menyimpan nilai data sekaligus membangun computation graph (graf komputasi) untuk melacak setiap operasi matematika yang terjadi, sehingga turunan (gradien) dapat dihitung secara otomatis (Automatic Differentiation).

```
class Tensor:
  def __init__(self, data, _children=(), requires_grad=True):
    self.data          = np.array(data, dtype=float)
    self.grad          = np.zeros_like(self.data)
    self._backward     = lambda: None
    self._prev         = set(_children)
    self.requires_grad = requires_grad
```

Deskripsi Atribut:

- data (numpy array): Nilai numerik aktual yang disimpan di dalam tensor.
- grad (numpy array): Nilai gradien/turunan dari tensor tersebut yang ukurannya sama dengan data.
- _backward (function): Fungsi yang mendefinisikan aturan rantai (chain rule) lokal untuk operasi spesifik yang menghasilkan tensor tersebut.
- _prev (set): Himpunan tensor parent (input) yang menghasilkan tensor saat ini (untuk membangun graf komputasi).
- requires_grad (boolean): Penanda apakah tensor ini memerlukan komputasi gradien saat proses propagasi mundur.

Deskripsi Method Utama:

- Operasi Matematika Dasar (__add__, __mul__, __matmul__, dll): Magic methods (overloading) yang memungkinkan operasi aritmatika langsung antar objek Tensor, sekaligus merangkai graf komputasi.
- Fungsi Aktivasi (relu, sigmoid, softmax, dll): Melakukan transformasi non-linear pada data tensor dan mendefinisikan turunan lokal dari fungsi aktivasi tersebut.
- backward(): Menginisiasi proses propagasi mundur (backpropagation) dengan melakukan Topological Sort untuk menghitung gradien semua node di dalam graf komputasi.
- zero_grad(): Mereset nilai atribut grad menjadi nol sebelum iterasi training berikutnya.

2.1.1.2 Kelas Dense (src/neuron/layer.py)

Kelas Dense mewarisi kelas Layer dan merepresentasikan satu buah hidden layer atau output layer yang bersifat fully-connected (terhubung penuh). Kelas ini melakukan transformasi linear matematis: y = xW + b.

```
class Dense(Layer):
  def __init__(self, in_features, out_features, use_bias=True,
               initializer='xavier_normal', init_kwargs=None):
    """Initialize a fully connected (Dense) layer."""
    self.in_features  = in_features
    self.out_features = out_features
    self.use_bias     = use_bias

    init_fn = get_initializer(initializer)
    kwargs  = init_kwargs or {}

    W_data = init_fn((in_features, out_features), **kwargs)
    self.W = Tensor(W_data)

    if use_bias:
      self.b = Tensor(np.zeros(out_features))
    else:
      self.b = None
```

Deskripsi Atribut:

- in_features (int): Jumlah neuron (fitur) dari layer sebelumnya yang masuk ke layer ini.
- out_features (int): Jumlah neuron pada layer ini.
- W (Tensor): Matriks bobot (weight) berukuran (in_features, out_features) yang akan diperbarui selama training.
- b (Tensor): Vektor bias berukuran (out_features,) (opsional).

Deskripsi Method Utama:

- forward(x): Melakukan operasi komputasi maju perkalian matriks input x dengan bobot W (x @ W) lalu ditambahkan dengan bias b.
- parameters(): Mengembalikan daftar (list) parameter yang dapat dilatih (trainable parameters), yaitu tensor W dan b pada layer tersebut.

2.1.1.3 Kelas FFNN (src/models/ffnn.py)

Kelas FFNN adalah kelas tingkat atas (high-level) yang merepresentasikan arsitektur Neural Network secara utuh. Kelas ini bertugas merangkai sekumpulan layer dan mengelola aliran data dari input hingga output.

```
class FFNN:
  def __init__(self, layer_sizes, activations, initializer='xavier_normal',
              init_kwargs=None, use_bias=True, use_rmsnorm=False):
    assert len(activations) == len(layer_sizes) - 1, \

    self.layer_sizes      = layer_sizes
    self.activation_names = activations
    self.layers           = []
    self.dense_layers     = []

    self._build(layer_sizes, activations, initializer, init_kwargs or {},
              use_bias, use_rmsnorm)
```

Deskripsi Atribut:

- layer_sizes (list of ints): Daftar yang merepresentasikan jumlah neuron pada setiap urutan layer (dari input ke output).
- activation_names (list of str): Daftar nama fungsi aktivasi yang digunakan di setiap transisi antar layer.
- layers (list): Kumpulan objek layer secara utuh (mencakup objek Dense, objek Aktivasi, dan objek Normalisasi jika ada) secara berurutan.
- dense_layers (list): Kumpulan objek khusus dari kelas Dense untuk mempermudah ekstraksi parameter berbobot.

Deskripsi Method Utama:

- _build(...): Method internal untuk menginisialisasi arsitektur jaringan secara sekuensial berdasarkan layer_sizes dan menyelipkan fungsi aktivasi serta RMSNorm.
- forward(x): Menjalankan data input x melewati semua layer di dalam self.layers secara berurutan. Outputnya tetap berupa objek Tensor.
- predict(x): Menjalankan forward(x) namun hanya mengembalikan nilai numpy array-nya (tanpa tracking gradien).
- parameters(): Mengumpulkan dan mengembalikan semua parameter yang dapat dilatih dari seluruh layers.
- plot_weight_distribution() & plot_gradient_distribution(): Method untuk memvisualisasikan histogram nilai bobot dan gradien per layer menggunakan matplotlib.
- save(path) & load(path): Method untuk menyimpan bobot model ke dalam file .npz dan memuatnya kembali.

2.1.1.4 Kelas-Kelas Aktivasi Fungsi (src/neuron/activations.py)

Kelas-kelas ini murni memanggil fungsi (method) dari **objek Tensor** yang menjadi inputnya (x).

```python
class Linear(Layer):
  def forward(self, x):
    return x

class ReLU(Layer):
  def forward(self, x):
    return x.relu()

class Sigmoid(Layer):
  def forward(self, x):
    return x.sigmoid()

class Tanh(Layer):
  def forward(self, x):
    return x.tanh()

class Softmax(Layer):
  def __init__(self, axis=-1):
    self.axis = axis
  def forward(self, x):
    return x.softmax(axis=self.axis)

class ELU(Layer):
  def __init__(self, alpha=1.0):
    self.alpha = alpha
  def forward(self, x):
    return x.elu(self.alpha)

class LeakyReLU(Layer):
  def __init__(self, alpha=0.01):
    self.alpha = alpha

  def forward(self, x):
    return x.leaky_relu(self.alpha)
```

- Linear: Ini adalah fungsi identitas (merespons dengan nilai input apa adanya). Biasanya digunakan pada output layer untuk kasus regresi. Kodenya hanya mengembalikan x tanpa perubahan.

- ReLU (Rectified Linear Unit): Memanggil x.relu(). Fungsi ini akan mengubah semua nilai negatif menjadi 0, dan membiarkan nilai positif apa adanya (max(0, x)). Ini adalah aktivasi paling umum untuk hidden layer.
- Sigmoid: Memanggil x.sigmoid(). Mengubah rentang nilai input menjadi antara 0 hingga 1. Sangat cocok untuk output layer pada klasifikasi biner.
- Tanh (Hyperbolic Tangent): Memanggil x.tanh(). Mirip dengan sigmoid, tetapi mengubah rentang nilai menjadi -1 hingga 1.
- Softmax: Fungsi ini digunakan untuk output layer pada klasifikasi multikelas (mengubah output menjadi distribusi probabilitas yang totalnya 1). Kelas ini menerima parameter axis (biasanya axis=-1 untuk menghitung per baris data dalam satu batch) sebelum meneruskannya ke x.softmax(axis=self.axis).
- ELU (Exponential Linear Unit): Memiliki parameter alpha (default 1.0). Berbeda dengan ReLU yang memotong nilai negatif menjadi 0, ELU memberikan kurva eksponensial halus untuk nilai negatif. Kelas ini akan meneruskan komputasinya ke x.elu(self.alpha).
- LeakyReLU: Memiliki parameter alpha (default 0.01). Untuk mengatasi masalah "dying ReLU" (neuron mati karena nilainya terus-menerus 0), LeakyReLU membiarkan sedikit "kebocoran" nilai negatif (dikalikan dengan alpha). Meneruskan komputasi ke x.leaky_relu(self.alpha).

2.1.1.5 Kelas Layer (src/neuron/base.py)

Kelas Layer adalah Abstract Base Class (ABC) yang menjadi fondasi dan parent class (kelas induk) bagi semua jenis layer dan fungsi aktivasi di dalam model jaringan. Dengan adanya kelas dasar ini, arsitektur model memiliki antarmuka (interface) yang seragam, sehingga objek layer komputasi (seperti Dense) dan objek stateless (seperti ReLU) dapat diproses secara seragam di dalam sebuah list pada kelas FFNN.

```
class Layer(ABC):
  @abstractmethod
  def forward(self, x):
  def __call__(self, x):
    return self.forward(x)
  def parameters(self):
    return []
```

Deskripsi Method Utama:

- forward(self, x): Sebuah @abstractmethod yang mendefinisikan proses aliran komputasi maju (forward pass). Karena bersifat abstrak, setiap kelas turunan (seperti Dense atau Sigmoid) wajib mengimplementasikan

atau menimpa (override) method ini dengan logika komputasi mereka masing-masing.

- __call__(self, x): Magic method yang memungkinkan sebuah instansiasi objek dari turunan kelas ini dapat dipanggil langsung layaknya sebuah fungsi biasa (contoh: layer(x) akan otomatis mengeksekusi layer.forward(x)).
- parameters(self): Method yang mengembalikan daftar parameter berbobot (seperti matriks bobot dan bias) dari sebuah layer. Secara default, method ini mengembalikan list kosong []. Hal ini sangat efisien karena fungsi aktivasi (yang tidak memiliki bobot yang bisa dilatih) tidak perlu mendefinisikan ulang method ini, sementara layer seperti Dense dapat menimpanya (override) untuk mengembalikan bobot W dan bias b.

2.1.1.6 Kelas RMSNorm (src/neuron/normalization.py)

Kelas RMSNorm mewarisi kelas Layer dan mengimplementasikan metode Root Mean Square Normalization. Normalisasi ini digunakan untuk menstabilkan dan mempercepat proses pelatihan model dengan cara menskalakan nilai aktivasi pada suatu layer menggunakan nilai Root Mean Square (RMS) dari input tersebut. Kelas ini merepresentasikan fitur bonus yang diimplementasikan sebagai alternatif yang lebih efisien dibandingkan Layer Normalization standar.

```
class RMSNorm(Layer):
  def __init__(self, features, eps=1e-8):
    self.features = features
    self.eps      = eps
    self.gamma    = Tensor(np.ones(features))

  def forward(self, x):
    rms = ((x * x).mean(axis=-1, keepdims=True) + self.eps)** 0.5
    return (x / rms) * self.gamma

  def parameters(self):
    return [self.gamma]
```

Deskripsi Atribut:

- features (int): Jumlah fitur (atau jumlah neuron) dari layer yang akan dinormalisasi.
- eps (float): Nilai epsilon yang sangat kecil (default 10^{-8}) yang ditambahkan ke dalam perhitungan akar kuadrat untuk menghindari pembagian dengan nol (numerical stability).
- gamma (Tensor): Parameter skala yang dapat dilatih (learnable scale parameter). Parameter ini diinisialisasi dengan nilai 1 (np.ones) untuk

setiap fitur, dan bobotnya akan terus diperbarui selama proses backpropagation.

Deskripsi Method Utama:

- forward(self, x): Menghitung nilai Root Mean Square dari input x pada dimensi fitur (axis=-1). Nilai input x kemudian dibagi dengan nilai rms tersebut, lalu dikalikan dengan parameter bobot gamma. Komputasi ini memanfaatkan method matematika dari kelas Tensor sehingga gradiennya tetap terawasi secara otomatis.
- parameters(self): Mengembalikan daftar parameter yang dapat dilatih dari layer ini. Karena RMSNorm tidak menggunakan bias (berbeda dengan LayerNorm biasa), method ini hanya mengembalikan sebuah list berisi tensor gamma ([self.gamma]).

2.1.1.7 Kelas Adam (src/optim/adam.py)

Kelas Adam adalah implementasi algoritma optimisasi Adaptive Moment Estimation. Kelas ini bertanggung jawab untuk memperbarui parameter bobot (W) dan bias (b) dari model Feedforward Neural Network menggunakan perhitungan gradien yang telah dihasilkan dari proses backpropagation. Berbeda dengan gradient descent standar yang menggunakan learning rate tetap untuk semua parameter, Adam menyimpan estimasi rata-rata pergerakan eksponensial dari gradien (momen pertama) dan kuadrat gradien (momen kedua) untuk menyesuaikan learning rate secara adaptif pada setiap parameter.

```
class Adam:
  def __init__(self, parameters, learning_rate=0.001, beta1=0.9, beta2=0.999, eps=1e-8,
l1_lambda=0.0, l2_lambda=0.0):
      self.parameters = parameters
      self.lr        = learning_rate
      self.beta1     = beta1
      self.beta2     = beta2
      self.eps       = eps
      self.l1_lambda  = l1_lambda
      self.l2_lambda  = l2_lambda
      self.t         = 0
      self.m = [np.zeros_like(p.data) for p in parameters]
      self.v = [np.zeros_like(p.data) for p in parameters]

  def step(self):
      self.t += 1
      for i, p in enumerate(self.parameters):
          grad = p.grad.copy()
```

```
        if self.l2_lambda > 0:
            grad += self.l2_lambda * p.data
        if self.l1_lambda > 0:
            grad += self.l1_lambda * np.sign(p.data)

        self.m[i] = self.beta1 * self.m[i] + (1.0 - self.beta1) * grad
        self.v[i] = self.beta2 * self.v[i] + (1.0 - self.beta2) * grad ** 2

        m_hat = self.m[i] / (1.0 - self.beta1 ** self.t)
        v_hat = self.v[i] / (1.0 - self.beta2 ** self.t)

        p.data -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)

    def zero_grad(self):
        for p in self.parameters:
            p.grad = np.zeros_like(p.data)
```

Deskripsi Atribut:

- parameters (list of Tensors): Daftar parameter model (seperti bobot dan bias dari setiap layer) yang akan dioptimasi.
- lr (float): Learning rate atau ukuran langkah dasar untuk pembaruan bobot.
- beta1 & beta2 (float): Koefisien peluruhan (decay rates) untuk menghitung estimasi momen pertama (beta1) dan momen kedua (beta2).
- eps (float): Nilai konstan epsilon yang sangat kecil untuk mencegah pembagian dengan nol saat pembaruan bobot (numerical stability).
- l1_lambda & l2_lambda (float): Parameter penalti untuk regularisasi L1 (Lasso) dan L2 (Ridge) guna mengurangi overfitting.
- t (int): Penghitung iterasi/langkah (timestep counter) yang digunakan dalam proses koreksi bias (bias correction).
- m & v (list of numpy arrays): Vektor yang menyimpan nilai estimasi momen pertama (rata-rata pergerakan gradien) dan momen kedua (rata-rata pergerakan kuadrat gradien) untuk setiap parameter.

Deskripsi Method Utama:

- step(self): Method utama yang dipanggil pada setiap iterasi pelatihan untuk memperbarui bobot parameter. Method ini pertama-tama menambahkan penalti regularisasi L1/L2 pada gradien (jika ada). Kemudian, method menghitung rata-rata bergerak gradien (self.m) dan kuadrat gradien (self.v). Setelah mengaplikasikan koreksi bias untuk mengkompensasi inisialisasi awal yang bernilai nol (m_hat dan v_hat),

method ini memodifikasi data aktual pada parameter (p.data) menggunakan persamaan pembaruan Adam.

- zero_grad(self): Method praktis untuk mereset seluruh nilai gradien (p.grad) pada setiap parameter di dalam list menjadi nol sebelum memulai proses forward dan backward pass pada iterasi atau batch berikutnya.

2.1.1.8 Kelas SGD (src/optim/gradient_descent.py)

Kelas SGD (Stochastic Gradient Descent) adalah implementasi dari algoritma optimisasi standar yang bertugas memperbarui parameter jaringan (bobot dan bias) berdasarkan nilai gradien yang dihitung selama proses backward propagation. Kelas ini juga mendukung penerapan teknik regularisasi L1 dan L2 secara langsung pada saat pembaruan gradien untuk mencegah overfitting.

```
class SGD:
  def __init__(self, parameters, learning_rate=0.01, l1_lambda=0.0, l2_lambda=0.0):
    self.parameters = parameters
    self.lr         = learning_rate
    self.l1_lambda  = l1_lambda
    self.l2_lambda  = l2_lambda

  def step(self):
    for p in self.parameters:
      grad = p.grad.copy()
      if self.l2_lambda > 0:
        grad += self.l2_lambda * p.data
      if self.l1_lambda > 0:
        grad += self.l1_lambda * np.sign(p.data)
      p.data -= self.lr * grad

  def zero_grad(self):
    for p in self.parameters:
      p.grad = np.zeros_like(p.data)
```

Deskripsi Atribut:

- parameters (list of Tensors): Daftar parameter model yang dapat dilatih dan akan diperbarui.
- lr (float): Learning rate (alpha) yang menentukan seberapa besar langkah pembaruan bobot.
- l1_lambda & l2_lambda (float): Nilai penalti (hyperparameter) untuk regularisasi L1 dan L2.

Deskripsi Method Utama:

- step(self): Melakukan iterasi pada setiap parameter untuk memperbarui nilainya. Jika l2_lambda atau l1_lambda lebih besar dari 0, method ini akan menambahkan nilai penalti ke dalam gradien. Setelah itu, nilai parameter diperbarui menggunakan rumus gradient descent: W_{new} = W_{old} - alpha times grad.
- zero_grad(self): Mereset atribut grad dari semua parameter menjadi nol agar tidak terakumulasi pada iterasi selanjutnya.

2.1.1.9 Modul Inisialisasi Bobot (src/optim/initializers.py)

Modul ini tidak diimplementasikan sebagai sebuah kelas (OOP), melainkan sebagai sekumpulan fungsi murni (pure functions) yang mengembalikan numpy array berisi nilai awal untuk matriks bobot. Pendekatan ini dipilih agar inisialisasi lebih fleksibel.

```
def zero_init(shape):
  return np.zeros(shape)

def uniform_init(shape, lower=-0.1, upper=0.1, seed=None):
  rng = np.random.default_rng(seed)
  return rng.uniform(lower, upper, shape)

def normal_init(shape, mean=0.0, variance=1.0, seed=None):
  rng = np.random.default_rng(seed)
  return rng.normal(mean, np.sqrt(variance), shape)

def xavier_uniform_init(shape, seed=None):
  # scale = sqrt(6 / (fan_in + fan_out))
  fan_in, fan_out = shape[0], shape[1]
  scale = np.sqrt(6.0 / (fan_in + fan_out))
  rng   = np.random.default_rng(seed)
  return rng.uniform(-scale, scale, shape)

def xavier_normal_init(shape, seed=None):
  # std = sqrt(2 / (fan_in + fan_out))
  fan_in, fan_out = shape[0], shape[1]
  std = np.sqrt(2.0 / (fan_in + fan_out))
  rng = np.random.default_rng(seed)
  return rng.normal(0.0, std, shape)

def he_uniform_init(shape, seed=None):
```

```python
    # scale = sqrt(6 / fan_in)
    fan_in = shape[0]
    scale  = np.sqrt(6.0 / fan_in)
    rng    = np.random.default_rng(seed)
    return rng.uniform(-scale, scale, shape)

def he_normal_init(shape, seed=None):
    # std = sqrt(2 / fan_in)
    fan_in = shape[0]
    std    = np.sqrt(2.0 / fan_in)
    rng    = np.random.default_rng(seed)
    return rng.normal(0.0, std, shape)

INIT_MAP = {
    'zero':          zero_init,
    'uniform':       uniform_init,
    'normal':        normal_init,
    'xavier_uniform': xavier_uniform_init,
    'xavier_normal':  xavier_normal_init,
    'he_uniform':     he_uniform_init,
    'he_normal':      he_normal_init,
}
def get_initializer(name):
    if name not in INIT_MAP:
        raise ValueError(f"Unknown initializer: '{name}'. Choose from {list(INIT_MAP)}")
    return INIT_MAP[name]
```

Daftar Fungsi Inisialisasi:

- zero_init: Menginisialisasi semua bobot dengan nilai 0.
- uniform_init: Menginisialisasi bobot dengan distribusi seragam (uniform) berdasarkan batas bawah dan batas atas tertentu.
- normal_init: Menginisialisasi bobot dengan distribusi normal (Gaussian) berdasarkan nilai mean dan variansi (variance).
- Bonus Inisialisasi (xavier_uniform, xavier_normal, he_uniform, he_normal): Implementasi inisialisasi tingkat lanjut (Xavier/Glorot dan He) yang menyesuaikan skala distribusi bobot berdasarkan jumlah neuron input (fan_in) dan output (fan_out). Ini berguna untuk mencegah masalah vanishing atau exploding gradient pada jaringan yang dalam.

Fungsi Bantuan:

- get_initializer(name): Sebuah factory function yang menerima string nama inisialisasi (contoh: 'xavier_normal') dan mengembalikan fungsi inisialisasi yang bersesuaian dengan memetakannya melalui dictionary INIT_MAP.

2.1.1.10 Modul Fungsi Loss (src/optim/loss.py)

Sama seperti initializers, modul ini berisi sekumpulan fungsi untuk menghitung nilai kesalahan (error/loss) antara hasil prediksi model dengan target asli (label). Fungsi-fungsi ini memanfaatkan operasi matematika dari kelas Tensor, sehingga komputasi gradiennya terhadap bobot jaringan terhubung ke dalam computation graph Autodiff.

```
def mse_loss(y_pred, y_true):
    if not isinstance(y_true, Tensor):
        y_true = Tensor(y_true, requires_grad=False)
    diff = y_pred - y_true
    return (diff * diff).mean()


def _clip_tensor(t, lo, hi):
    out = Tensor(np.clip(t.data, lo, hi), (t,))
    def _backward():
        t.grad += out.grad
    out._backward = _backward
    return out


def bce_loss(y_pred, y_true):
    if not isinstance(y_true, Tensor):
        y_true = Tensor(y_true, requires_grad=False)
    p = _clip_tensor(y_pred, _EPS, 1.0 - _EPS)
    return -(y_true * p.log() + (1.0 - y_true) * (1.0 -
p).log()).mean()


def cce_loss(y_pred, y_true):
    if not isinstance(y_true, Tensor):
        y_true = Tensor(y_true, requires_grad=False)
    p = _clip_tensor(y_pred, _EPS, 1.0)
    return -(y_true * p.log()).sum(axis=-1).mean()
```

[redacted]

Daftar Fungsi Loss:

- mse_loss(y_pred, y_true): Mengimplementasikan Mean Squared Error (MSE), yaitu rata-rata dari kuadrat selisih antara nilai prediksi dan nilai sebenarnya. Cocok untuk kasus regresi.
- bce_loss(y_pred, y_true): Mengimplementasikan Binary Cross-Entropy (BCE). Digunakan untuk klasifikasi biner dengan membandingkan probabilitas prediksi terhadap label 0 atau 1.
- cce_loss(y_pred, y_true): Mengimplementasikan Categorical Cross-Entropy (CCE) untuk kasus klasifikasi banyak kelas (multi-class).

Fungsi Bantuan / Stabilitas Numerik:

- _clip_tensor(t, lo, hi): Fungsi internal yang sangat penting untuk stabilitas komputasi. Saat menghitung BCE atau CCE, terdapat perhitungan logaritma (log(p)). Jika nilai probabilitas p bernilai mutlak 0, perhitungan log(0) akan menghasilkan error numerik (Not a Number/NaN) atau tak terhingga (infinity). Fungsi ini membatasi nilai sekecil mungkin ke batas _EPS (sebesar 10^{-12}) sebelum dilogaritmakan, sambil memastikan gradien pada backward pass tetap tersalurkan tanpa terputus.

2.1.2 Penjelasan forward propagation

```
def forward(self, x):
    t = x if isinstance(x, Tensor) else Tensor(x, requires_grad=False)
    for layer in self.layers:
        t = layer(t)
    return t
```

Forward propagation adalah proses mengalirkan data input dari layer pertama (input layer) menuju layer terakhir (output layer) secara berurutan. Pada setiap layer, dilakukan dua operasi: transformasi linear oleh kelas `Dense` dan transformasi non-linear oleh kelas aktivasi.

2.1.2.1 Alur Komputasi

Misalkan sebuah FFNN memiliki L layer dengan bobot W^l dan bias b^l. Untuk sebuah batch input X berukuran (B, input_features):

1. **Pre-activation (transformasi linear):** Pada layer ke-I, dihitung nilai pra-aktivasi $z^l = a^{l-1} W^l + b^l$ di mana a^{l-1} adalah output dari layer sebelumnya (atau X untuk layer pertama), dan W^l berukuran (input_features^l, output_features^l).

2. **Post-activation (transformasi non-linear:** Nilai pra-aktivasi ditransformasi dengan fungsi aktivasi yang dipilih: $a^l = f(z^l)$
3. **Proses berulang** I=1 hingga I=L sehingga output akhir model adalah $\hat{y} = a^L$.

2.1.2.2 Implementasi melalui Autodiff

Dalam implementasi ini, setiap operasi (__matmul__, __add__) pada objek Tensor secara otomatis mendaftarkan fungsi _backward yang sesuai dan menyimpan referensi ke node-node parent (_prev). Akibatnya, setiap pemanggilan model.forward(X) secara implisit membangun sebuah computation graph (directed acyclic graph) yang mencatat seluruh alur komputasi. Graf ini kemudian akan ditraversal saat backward pass dilakukan.

Implementasi pada kelas FFNN (method forward):

```
def forward(self, x):
        t = x if isinstance(x, Tensor) else Tensor(x,
requires_grad=False)
        for layer in self.layers:
            t = layer(t)
        return t
```

2.1.2.3 Peran RMSNorm dalam Forward Pass

Jika use_rmsnorm=True, setelah setiap layer Dense (sebelum aktivasi), output dinormalisasi menggunakan RMSNorm: $\hat{x} = (x / \text{sqrt}(\text{mean}(x^2) + \varepsilon)) \times \gamma$ Normalisasi ini menstabilkan skala aktivasi di setiap layer, mencegah exploding/vanishing gradients pada jaringan yang dalam.

2.1.3 Penjelasan backward propagation dan weight update

2.1.3.1 Automatic Differentiation *(Autodiff)*

Implementasi ini menggunakan pendekatan reverse-mode automatic differentiation (autodiff). Berbeda dengan backpropagation manual yang mengharuskan derivasi persamaan gradien secara eksplisit, autodiff memanfaatkan computation graph yang dibangun selama forward pass untuk secara otomatis menghitung gradien setiap parameter.

2.1.3.2 Proses Backward Pass

Setelah forward pass menghasilkan nilai loss, loss.backward() dipanggil untuk memuai propagasi mundur:

1. **Topological Sort:** Method backward() pada kelas Tensor membangun urutan topologis seluruh node di computation graph menggunakan DFS (Depth-First-Search). Ini memastikan setiap node diproses setelah semua node yang bergantung padanya.
2. **Seed Gradient:** Gradien dari node loss (output) diinisialisasi dengan nilai 1 (self.grad = np.ones_like(self.data)), sesuai prinsip $\partial L/\partial L = 1$.
3. **Propagasi Mundur:** Iterasi dilakukan dari node output menuju node input dalam urutan terbalik. Untuk setiap node, fungsi _backward() yang sudah didaftarkan saat forward pass dipanggil. Fungsi ini menghitung kontribusi gradien ke node-node parent menggunakan chain rule lokal.

Contoh pada operasi perkalian matriks (__matmul__):

```
def __matmul__(self, other):
    out = Tensor(self.data @ other.data, (self, other))
    def _backward():
        if self.requires_grad:
            self.grad  += out.grad @ other.data.T
        if other.requires_grad:
            other.grad += self.data.T @ out.grad
    out._backward = _backward
    return out
```

2.1.3.3 Chain Rule pada Graf Komputasi

Setiap operasi primitif (penjumlahan, perkalian, matmul, exp, log, dll.) mendefinisikan turunan lokal. Autodiff mengalikan turunan lokal tersebut dengan gradien yang datang dari node selanjutnya (upstream gradient), mengimplementasikan chain rule secara otomatis tanpa perlu menulis rumus backpropagation secara manual untuk setiap arsitektur.

Validasi implementasi ini dilakukan dengan *gradient check* numerik. Perbedaan maksimum antara gradien analitis (autodiff) dan gradien numerik (metode beda hingga) adalah $\mathbf{3.22 \times 10^{-11}}$, jauh di bawah batas toleransi 10^{-5}, membuktikan kebenaran implementasi autodiff.

2.1.3.4 Weight Update

Setelah gradien dihitung, optimizer memperbarui bobot. Untuk **SGD:**
$W_{e}w = W_{o}d - \alpha \cdot (\partial L/\partial W + \lambda_2 W + \lambda_1 \cdot \mathrm{sign}(W))$
Untuk Adam, weight update dilakukan dengan memanfaatkan momen adaptif:

- Momen pertama (running mean gradien): $m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t$
- Momen kedua (running variance gradien): $v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2$
- Update setelah bias correction: $W_{e}w = W_{o}d - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$

Adam lebih adaptif dibandingkan SGD karena setiap parameter memiliki learning rate efektif yang berbeda, disesuaikan dengan riwayat gradien parameter tersebut

2.1.3.5 Training Loop

Satu epoch terdiri dari:

1. Mengacak (shuffle) data training
2. Iterasi mini-batch: untuk setiap batch → zero_grad()→ forward()→ hitung loss → loss.backward() → optimizer.step()
3. Loggin rata-rata training loss dan validation loss

2.2 Hasil Pengujian

2.2.1 Preprocessing Dataset

Dataset Global Student Placement & Salary yang digunakan dalam eksperimen ini terdiri dari 10.000 sampel data dengan total 12 kolom, di mana 11 kolom merupakan fitur atribut dan 1 kolom (placement_status) merupakan target prediksi klasifikasi. Sebelum data dimasukkan ke dalam model jaringan saraf tiruan, serangkaian tahapan preprocessing (pra-pemrosesan) dilakukan untuk mengubah data mentah menjadi representasi numerik yang optimal bagi komputasi model. Setelah seluruh tahapan selesai, dimensi fitur input terekspansi dari 11 menjadi 24 fitur.

```
_COLLEGE_TIER_MAP = {'Tier 1': 3, 'Tier 2': 2, 'Tier 3': 1}
_RANK_BAND_MAP    = {'Top 100': 4, '100-300': 3, '300+': 2}

_ONEHOT_COLS = ['country', 'specialization', 'industry']

_NUM_COLS = [
  'cgpa', 'backlogs', 'internship_count',
  'aptitude_score', 'communication_score', 'internship_quality_score',
]

def load_and_preprocess(csv_path):
  df = pd.read_csv(csv_path)

  y = (df['placement_status'] == 'Placed').astype(float).values.reshape(-1, 1)

  df['college_tier']           = df['college_tier'].map(_COLLEGE_TIER_MAP)
  df['university_ranking_band'] = df['university_ranking_band'].map(_RANK_BAND_MAP)

  df = pd.get_dummies(df, columns=_ONEHOT_COLS, dtype=float)
  df = df.drop(columns=['placement_status'])
  feature_cols = df.columns.tolist()
```

```
    X = df.values.astype(float)

    n     = len(X)
    idx   = np.random.default_rng(42).permutation(n)
    n_train = int(0.70 * n)
    n_val   = int(0.15 * n)

    train_idx = idx[:n_train]
    val_idx   = idx[n_train:n_train + n_val]
    test_idx  = idx[n_train + n_val:]

    X_train, y_train = X[train_idx], y[train_idx]
    X_val,   y_val   = X[val_idx],   y[val_idx]
    X_test,  y_test  = X[test_idx],  y[test_idx]

    num_indices = [feature_cols.index(c) for c in _NUM_COLS if c in feature_cols]
    mean = X_train[:, num_indices].mean(axis=0)
    std  = X_train[:, num_indices].std(axis=0) + 1e-8

    X_train[:, num_indices] = (X_train[:, num_indices] - mean) / std
    X_val[:,   num_indices] = (X_val[:,   num_indices] - mean) / std
    X_test[:,  num_indices] = (X_test[:,  num_indices] - mean) / std

    return X_train, X_val, X_test, y_train, y_val, y_test, feature_cols

def accuracy(y_pred_data, y_true):
    preds = (y_pred_data >= 0.5).astype(float)
    return float((preds == y_true).mean())
```

Berikut adalah rincian tahapan preprocessing yang dilakukan:

1. Penanganan Nilai Kosong (Missing Values):
   Eksplorasi awal pada dataset menunjukkan bahwa tidak ada data yang hilang (missing values) pada keseluruhan 10.000 sampel. Oleh karena itu, tahapan imputasi data (pengisian nilai kosong) tidak diperlukan.
2. Pengkodean Data Kategorikal Berurutan (Ordinal Encoding):
   Fitur kategorikal yang memiliki makna hierarki atau tingkatan, yaitu college_tier dan university_ranking_band, ditransformasikan menggunakan representasi angka ordinal (misalnya 1, 2, 3). Hal ini dilakukan agar model dapat memahami hubungan urutan besaran atau prioritas dari tingkatan institusi tersebut.
3. Pengkodean Data Kategorikal Nominal (One-Hot Encoding):

Fitur kategorikal yang tidak memiliki tingkatan urutan secara natural, yaitu country, specialization, dan industry, diubah menggunakan teknik One-Hot Encoding. Teknik ini merepresentasikan setiap kategori unik sebagai kolom biner baru (bernilai 0 atau 1). Pendekatan ini dipilih untuk mencegah model memberikan bobot lebih tinggi pada suatu kategori hanya karena direpresentasikan dengan angka numerik yang lebih besar. Transformasi inilah yang membuat jumlah fitur input bertambah menjadi 24 fitur.

4. Normalisasi Data Numerik (Standardization):
   Enam fitur numerik (yaitu cgpa, backlogs, internship_count, aptitude_score, communication_score, dan internship_quality_score) memiliki rentang nilai yang sangat bervariasi. Agar gradien tidak meledak (exploding) dan konvergensi loss lebih cepat selama pelatihan, fitur-fitur ini distandardisasi menggunakan metode Zero Mean Unit Variance. Parameter standardisasi (mean dan standar deviasi) hanya dihitung (di-fit) pada data training, lalu diaplikasikan ke data validation dan testing. Hal ini krusial untuk mencegah terjadinya data leakage (kebocoran informasi dari data uji ke dalam proses pelatihan).
5. Pembagian Dataset (Data Splitting):
   Untuk mengevaluasi performa model secara objektif, keseluruhan dataset dibagi menjadi tiga bagian (subset) yang saling lepas dengan rasio 70:15:15, yaitu:
   - Data Training (70% / 7.000 sampel): Digunakan untuk melatih model dan memperbarui bobot parameter.
   - Data Validation (15% / 1.500 sampel): Digunakan untuk mengevaluasi loss pada setiap epoch (hyperparameter tuning) tanpa memengaruhi bobot.
   - Data Testing (15% / 1.500 sampel): Digunakan sebagai ujian akhir (data yang sama sekali belum pernah dilihat model) untuk mengukur metrik akurasi final.

2.2.2 Pengaruh depth dan width

Dalam arsitektur Feedforward Neural Network (FFNN), kedalaman (depth) merujuk pada jumlah total hidden layer yang digunakan, sedangkan lebar (width) merujuk pada jumlah neuron yang terdapat di dalam masing-masing hidden layer. Pada implementasi kelas FFNN yang telah dibangun, kedua hyperparameter ini dikonfigurasi melalui argumen layer_sizes.

Eksperimen ini bertujuan untuk menganalisis bagaimana variasi kapasitas representasi model (melalui depth dan width) mempengaruhi hasil akhir prediksi, grafik loss, serta distribusi bobot dan gradien pada dataset Global Student Placement & Salary. Semua model pada eksperimen ini dilatih selama 100 epoch menggunakan learning rate 0.001, batch size 64 dan dengan fungsi aktivasi ReLu pada semua hidden layer dan Sigmoid pada output layer.

2.2.2.1 Eksperimen Variasi Lebar (Width) dengan Kedalaman Tetap

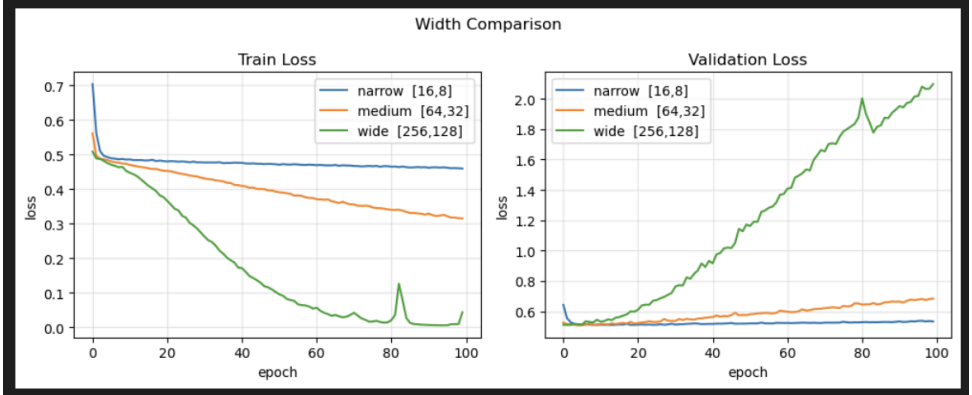
```
width_configs = {
```

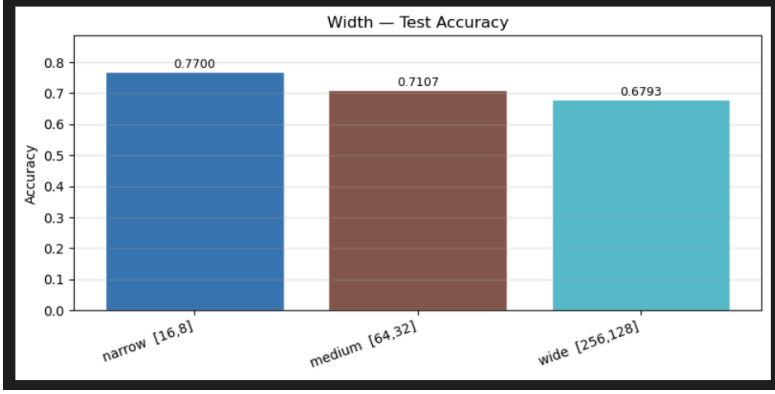
```
  'narrow  [16,8]':  [N_FEATURES, 16, 8, 1],
  'medium  [64,32]': [N_FEATURES, 64, 32, 1],
  'wide  [256,128]': [N_FEATURES, 256, 128, 1],
}
width_histories, width_accs = {}, {}

for name, sizes in width_configs.items():
  model = FFNN(sizes, ['relu'] * (len(sizes) - 2) + ['sigmoid'])
  opt   = Adam(model.parameters(), learning_rate=LR)
  hist  = train(model, X_train, y_train, opt, bce_loss,
              epochs=EPOCHS, batch_size=BS, X_val=X_val, y_val=y_val, verbose=0)
  acc   = accuracy(model.predict(X_test), y_test)
  width_histories[name], width_accs[name] = hist, acc
  print(f'{name}  test_acc={acc:.4f}')

compare_histories(list(width_histories.values()), list(width_histories.keys()), title='Width Comparison')
bar_comparison(list(width_accs.keys()), list(width_accs.values()), title='Width — Test Accuracy')
```

```
narrow  [16,8]  test_acc=0.7700
medium  [64,32]  test_acc=0.7107
wide  [256,128]  test_acc=0.6793
```





Pada skenario ini, jumlah hidden layer ditetapkan sebanyak 2 hidden layer, sementara jumlah neuron pada setiap layer divariasikan menjadi tiga konfigurasi:

- Model W1 (Narrow): layer_sizes = [24, 16, 8, 1]
- Model W2 (Medium): layer_sizes = [24, 64, 32, 1]
- Model W3 (Wide): layer_sizes = [24, 256, 128, 1]

Analisis Hasil Prediksi dan Loss:

Berdasarkan visualisasi menggunakan method plot_weight_distribution() dan plot_gradient_distribution(), dapat diamati bahwa seiring bertambahnya lebar jaringan, distribusi nilai bobot cenderung lebih menyebar (rentang nilai lebih lebar) dibandingkan model narrow yang bobotnya terpusat lebih rapat di sekitar nol. Gradien pada Model W3 (Wide) terlihat memiliki variansi yang lebih besar antar neuron dibandingkan W1, yang mengindikasikan setiap neuron mencoba mengoptimalkan fitur yang lebih spesifik. Namun demikian, besarnya gradien tidak secara otomatis menghasilkan generalisasi yang lebih baik, terbukti dari akurasi test W3 yang justru paling rendah.

Analisis Distribusi Bobot dan Gradien:

Berdasarkan visualisasi menggunakan method plot_weight_distribution() dan plot_gradient_distribution(), dapat diamati bahwa seiring bertambahnya lebar jaringan, distribusi nilai bobot cenderung (misal: lebih menyebar/lebih terpusat di nol). Gradien pada Model W3 terlihat (misal: memiliki variansi yang lebih besar dibandingkan W1, yang mengindikasikan setiap neuron mencoba mempelajari fitur yang sangat spesifik).

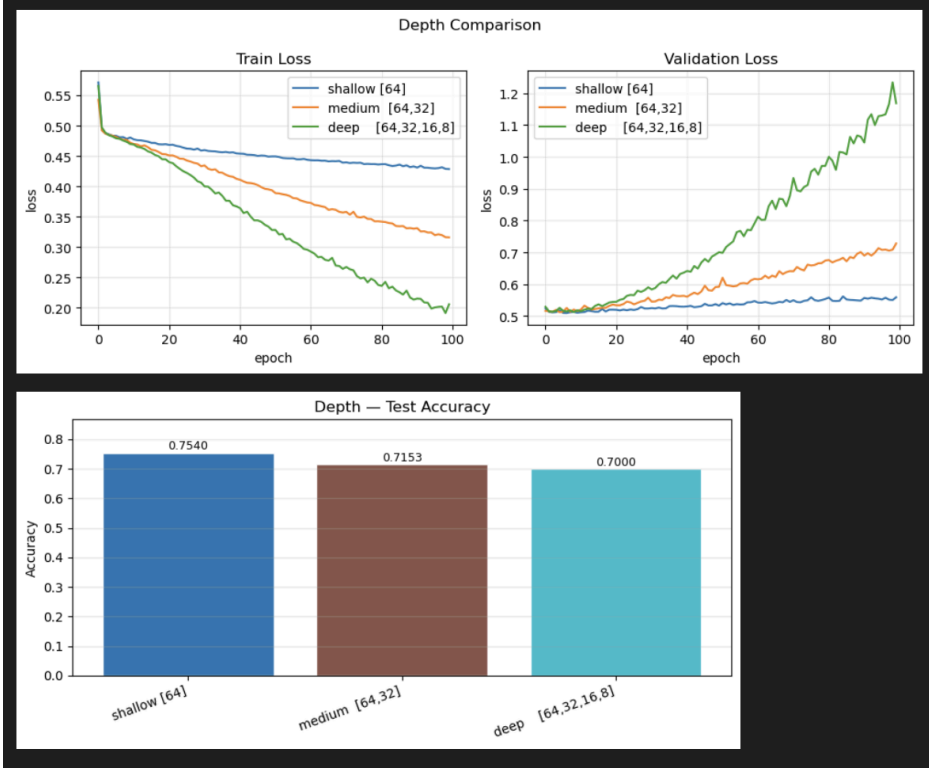
2.2.2.2 Eksperimen Variasi Kedalaman (Depth) dengan Lebar Tetap

```
depth_configs = {
  'shallow [64]':        [N_FEATURES, 64, 1],
  'medium  [64,32]':     [N_FEATURES, 64, 32, 1],
  'deep    [64,32,16,8]': [N_FEATURES, 64, 32, 16, 8, 1],
}
depth_histories, depth_accs = {}, {}

for name, sizes in depth_configs.items():
  model = FFNN(sizes, ['relu'] * (len(sizes) - 2) + ['sigmoid'])
  opt   = Adam(model.parameters(), learning_rate=LR)
  hist  = train(model, X_train, y_train, opt, bce_loss,
              epochs=EPOCHS, batch_size=BS, X_val=X_val, y_val=y_val, verbose=0)
  acc   = accuracy(model.predict(X_test), y_test)
  depth_histories[name], depth_accs[name] = hist, acc
  print(f'{name}  test_acc={acc:.4f}')
```

```
compare_histories(list(depth_histories.values()), list(depth_histories.keys()), title='Depth Comparison')
bar_comparison(list(depth_accs.keys()), list(depth_accs.values()), title='Depth — Test Accuracy')
```

```
shallow [64]  test_acc=0.7540
medium  [64,32]  test_acc=0.7153
deep    [64,32,16,8]  test_acc=0.7000
```



Pada skenario ini, jumlah neuron pada hidden layer pertama ditetapkan sebanyak 64 neuron, sementara jumlah hidden layer divariasikan:

- Model D1 (Shallow): layer_sizes = [24, 64, 1] (1 Hidden Layer)
- Model D2 (Medium): layer_sizes = [24, 64, 32, 1] (2 Hidden Layer)
- Model D3 (Deep): layer_sizes = [24, 64, 32, 16, 8, 1] (4 Hidden Layer)

Analisis Hasil Prediksi dan Loss:

Akurasi yang diperoleh berturut-turut adalah Model D1 sebesar 75.40% , Model D2 sebesar 71.53% , dan Model D3 sebesar 70.00%. Membandingkan grafik loss dari ketiga model, penambahan kedalaman jaringan pada dataset ini justru menurunkan performa. Model D1 (shallow) berhasil mempertahankan validation loss yang rendah dan stabil. Namun, pada Model D3 (jaringan paling dalam), terlihat bahwa model langsung mengalami overfitting yang ekstrim sejak awal epoch, ditandai dengan validation loss yang melonjak tajam ke atas.

Analisis Distribusi Bobot dan Gradien:

Pengamatan pada distribusi gradien menunjukkan perbedaan yang signifikan. Pada Model D3 (jaringan terdalam), distribusi gradien pada layer-layer awal (dekat input) cenderung sangat kecil mendekati nol dibandingkan layer mendekati output. Hal ini menunjukkan efek pelemahan gradien (vanishing gradient) secara natural ketika sinyal kesalahan dipropagasikan mundur melewati banyak layer. Sebaliknya, pada Model D1, distribusi gradien antar layer relatif lebih seimbang dan stabil.

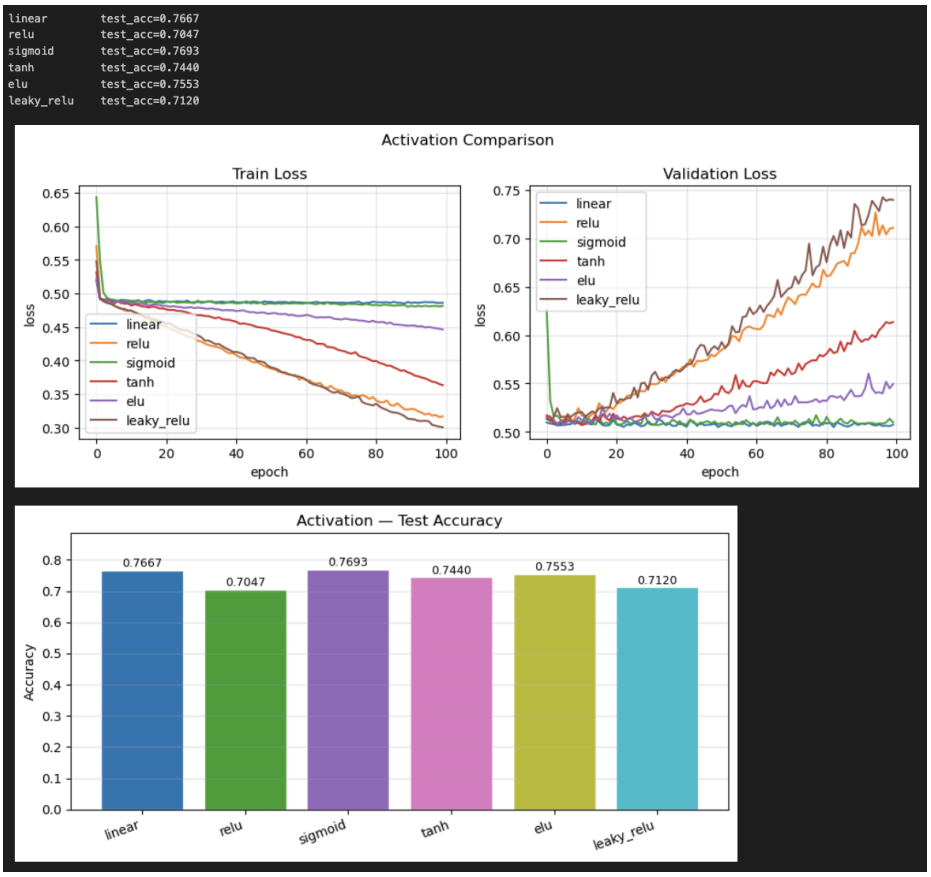
2.2.3 Pengaruh fungsi aktivasi

Pengujian ini bertujuan melihat pengaruh fungsi aktivasi pada kedua hidden layer dari total 3 layer (2 hidden + 1 output) arsitektur dasar yang kami gunakan, yaitu [24, 64, 32, 1]. Pada eksperimen ini, fungsi aktivasi yang sama diterapkan pada seluruh hidden layer secara bersamaan, sedangkan output layer selalu menggunakan Sigmoid.

```
act_configs  = ['linear', 'relu', 'sigmoid', 'tanh', 'elu', 'leaky_relu']
act_histories, act_accs = {}, {}

for act in act_configs:
  model = FFNN(BASE_SIZES, [act, act, 'sigmoid'])
  opt   = Adam(model.parameters(), learning_rate=LR)
  hist  = train(model, X_train, y_train, opt, bce_loss,
              epochs=EPOCHS, batch_size=BS, X_val=X_val, y_val=y_val, verbose=0)
  acc   = accuracy(model.predict(X_test), y_test)
  act_histories[act], act_accs[act] = hist, acc
  print(f'{act:12s}  test_acc={acc:.4f}')

compare_histories(list(act_histories.values()), list(act_histories.keys()), title='Activation
Comparison')
bar_comparison(list(act_accs.keys()), list(act_accs.values()), title='Activation — Test
Accuracy')
```



Analisis Hasil Prediksi dan Loss:
Berdasarkan hasil pengujian, model dengan fungsi aktivasi Sigmoid secara mengejutkan memberikan akurasi tertinggi sebesar 76.93% , diikuti oleh Linear (76.67%) dan ELU (75.53%). Sebaliknya, fungsi dari keluarga ReLU standar justru memberikan performa yang lebih rendah, dengan LeakyReLU menghasilkan akurasi 71.20% dan ReLU terendah di 70.47%.

Melihat grafik validation loss, fungsi Sigmoid dan Linear memiliki grafik yang sangat stabil dan datar. Sebaliknya, grafik validation loss untuk ELU, LeakyReLU, ReLU, dan Tanh melonjak sangat tajam seiring berjalannya epoch, mengindikasikan terjadinya overfitting yang parah pada fungsi-fungsi tersebut.

Kesimpulan:
Untuk memodelkan dataset Global Student Placement & Salary menggunakan arsitektur ini, fungsi aktivasi Sigmoid (atau Linear) adalah pilihan yang paling optimal karena mampu menjaga stabilitas validation loss tanpa terjebak dalam overfitting, sehingga menghasilkan akurasi pengujian yang paling tinggi.

2.2.4 Pengaruh learning rate

Arsitektur: [24, 64, 32, 1] dengan aktivasi ReLU dan Sigmoid. Optimizer yang digunakan adalah SGD (bukan Adam) agar perbedaan antar learning rate lebih terlihat. Dilatih 100 epoch, batch size 64.

```
lr_configs = [0.1, 0.01, 0.001]
lr_histories, lr_accs = {}, {}

for lr in lr_configs:
  model = FFNN(BASE_SIZES, ['relu', 'relu', 'sigmoid'])
  opt   = SGD(model.parameters(), learning_rate=lr)
  hist  = train(model, X_train, y_train, opt, bce_loss,
              epochs=EPOCHS, batch_size=BS, X_val=X_val, y_val=y_val, verbose=0)
  acc   = accuracy(model.predict(X_test), y_test)
  lr_histories[f'lr={lr}'], lr_accs[f'lr={lr}'] = hist, acc
  print(f'lr={lr}  test_acc={acc:.4f}')

compare_histories(list(lr_histories.values()), list(lr_histories.keys()), title='Learning Rate
(SGD)')
bar_comparison(list(lr_accs.keys()), list(lr_accs.values()), title='Learning Rate — Test
Accuracy')

lr=0.1  test_acc=0.7267
lr=0.01  test_acc=0.7700
lr=0.001 test_acc=0.7513
```

Analisis :

Tiga learning rate yang diuji memperlihatkan trade-off yang jelas antara kecepatan dan stabilitas konvergensi:

- lr=0.1 (terlalu besar): Konvergensi cepat di epoch-epoch awal, namun loss training berosilasi. Model tidak dapat fine-tune bobot secara halus sehingga akurasi akhirnya lebih rendah (72.67%). Pada grafik, terlihat loss tidak turun secara mulus.
- lr=0.01 (sedang): Konvergensi stabil dan cepat, memberikan performa terbaik (77.00%). Ukuran step cukup besar untuk belajar secara efektif, namun tidak terlalu besar sehingga melewati minimum loss.
- lr=0.001 (kecil): Konvergensi paling stabil namun lambat. Pada 100 epoch dengan SGD, model belum sepenuhnya konvergen (final train loss: 0.4908, val loss: 0.5142) sehingga akurasi akhirnya tidak optimal (75.13%).

Distribusi gradien bobot juga terpengaruh oleh learning rate: lr besar menghasilkan gradien yang lebih volatile, sedangkan lr kecil menghasilkan gradien yang lebih stabil namun lambat berubah.

2.2.5 Pengaruh inisialisasi bobot

Arsitektur: [24, 64, 32, 1] dengan aktivasi ReLU dan Sigmoid. Optimizer Adam (lr=0.001), 100 epoch, batch size 64. Setiap metode inisialisasi diuji dengan seed yang tidak ditetapkan untuk melihat karakteristik naturalnya.

Metode Inisialisasi	Test Accuracy
Zero	60.73%
Uniform [-0.1, 0.1]	74.33%
Normal (μ=0, σ^2=1)	71.80%
Xavier Uniform	71.87%
Xavier Normal	72.67%
He Uniform	73.20%
He Normal	71.87%

Analisis

- Zero initialization menghasilkan performa terburuk (60.73%). Seluruh neuron pada setiap layer memiliki bobot dan gradien yang identik (symmetry problem), sehingga semua neuron belajar fitur yang sama dan kapasitas model tidak dimanfaatkan secara penuh.
- Uniform initialization dengan rentang kecil [-0.1, 0.1] menghasilkan akurasi test terbaik (74.33%). Bobot awal yang kecil dan simetris menghasilkan gradien yang stabil di awal training tanpa risiko saturasi pada fungsi aktivasi.
- He initialization (uniform: 73.20%) memberikan performa kedua terbaik. He dirancang khusus untuk ReLU dengan skala $\sqrt{(6/f_i)}$, sehingga secara teoritis paling sesuai untuk arsitektur ini yang menggunakan ReLU di hidden layers.
- Xavier initialization (uniform: 71.87%, normal: 72.67%) dirancang untuk menjaga variance aktivasi konstan di seluruh layer, cocok untuk fungsi aktivasi seperti Sigmoid dan Tanh. Pada arsitektur yang menggunakan ReLU, Xavier sedikit kurang optimal dibanding He.
- Normal initialization dengan default (μ=0, σ^2=1) menghasilkan bobot awal yang terlalu besar sehingga memperlambat konvergensi (71.80%). Skala variance=1 tidak disesuaikan dengan ukuran layer, berbeda dari Xavier/He yang mempertimbangkan fan_in dan fan_out.

- Secara keseluruhan, semua metode kecuali Zero memberikan akurasi dalam rentang 71–74%, menunjukkan bahwa pemilihan initializer berpengaruh nyata namun tidak dramatis pada arsitektur kecil ini. Perbedaan performa lebih signifikan pada jaringan yang lebih dalam.

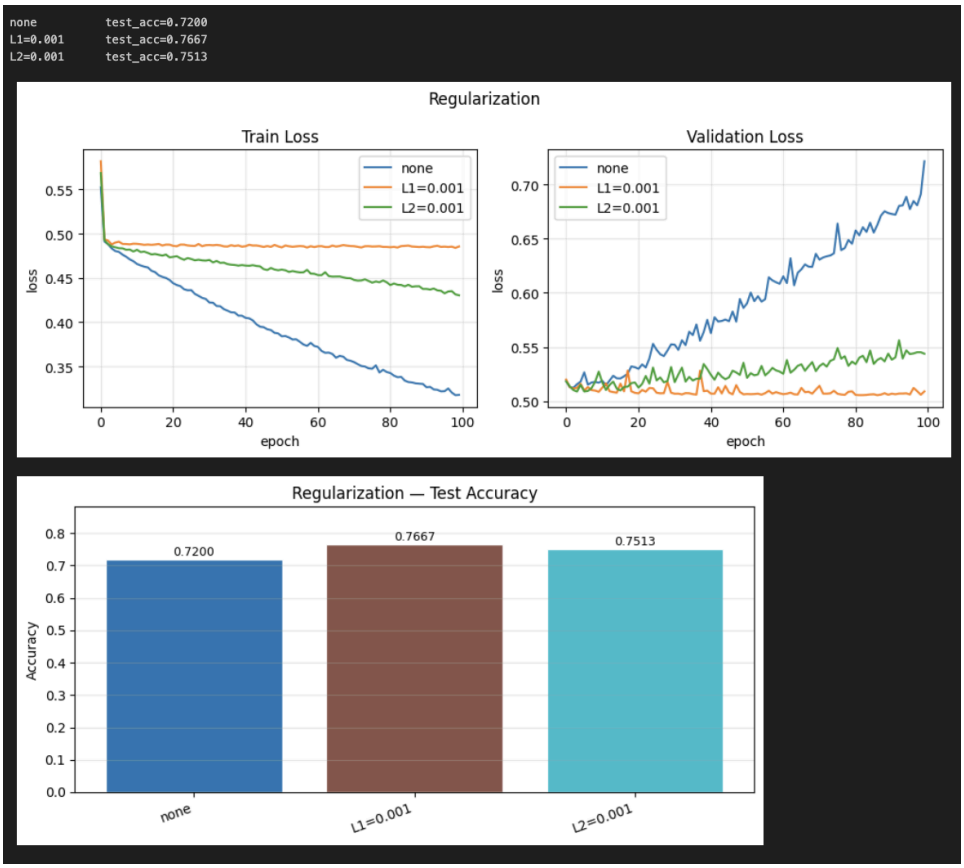
2.2.6 Pengaruh regularisasi

Arsitektur: [24, 64, 32, 1] dengan aktivasi ReLU dan Sigmoid. Optimizer Adam (lr=0.001), 100 epoch, batch size 64. Koefisien regularisasi: λ=0.001 untuk L1 dan L2.

```
reg_configs = [
  ('none',     dict(l1_lambda=0.0,   l2_lambda=0.0)),
  ('L1=0.001', dict(l1_lambda=0.001, l2_lambda=0.0)),
  ('L2=0.001', dict(l1_lambda=0.0,   l2_lambda=0.001)),
]
reg_histories, reg_accs = {}, {}

for name, reg_kwargs in reg_configs:
  model = FFNN(BASE_SIZES, ['relu', 'relu', 'sigmoid'])
  opt   = Adam(model.parameters(), learning_rate=LR, **reg_kwargs)
  hist  = train(model, X_train, y_train, opt, bce_loss,
              epochs=EPOCHS, batch_size=BS, X_val=X_val, y_val=y_val, verbose=0)
  acc   = accuracy(model.predict(X_test), y_test)
  reg_histories[name], reg_accs[name] = hist, acc
  print(f'{name:12s}  test_acc={acc:.4f}')

compare_histories(list(reg_histories.values()), list(reg_histories.keys()), title='Regularization')
bar_comparison(list(reg_accs.keys()), list(reg_accs.values()), title='Regularization — Test
Accuracy')
```



Konfigurasi	Test Accuracy
Tanpa Regulasi	72.00%
L1 (λ=0.001)	76.67%
L2 (λ=0.001)	75.13%

Analisis :
Kedua metode regularisasi secara signifikan meningkatkan performa dibandingkan model tanpa regularisasi, mengkonfirmasi bahwa model dasar mengalami overfitting:

- Tanpa regularisasi (72.00%): Model overfitting terhadap data training, training loss turun rendah tetapi validation loss mulai meningkat setelah beberapa epoch.
- L1 regularisasi (76.67%): Memberikan performa terbaik. L1 menambahkan penalti $\lambda \cdot |W|$ sehingga mendorong banyak bobot menuju tepat nol (sparse weights). Sparsifikasi ini bertindak sebagai feature selection implisit, sangat berguna pada dataset dengan banyak fitur yang kurang relevan (seperti one-hot encoded country/industry).
- L2 regularisasi (75.13%): Juga meningkatkan performa dibanding tanpa regularisasi, namun tidak sebaik L1 pada kasus ini. L2 menambahkan penalti

$\lambda \cdot W^2$ yang mendorong bobot kecil tetapi tidak nol (weight decay). L2 menyebarkan representasi lebih merata ke semua neuron.

Distribusi bobot model dengan L1 menunjukkan puncak yang jauh lebih tajam di sekitar nol dibandingkan L2 atau tanpa regularisasi, sesuai dengan sifat sparsifikasi L1.

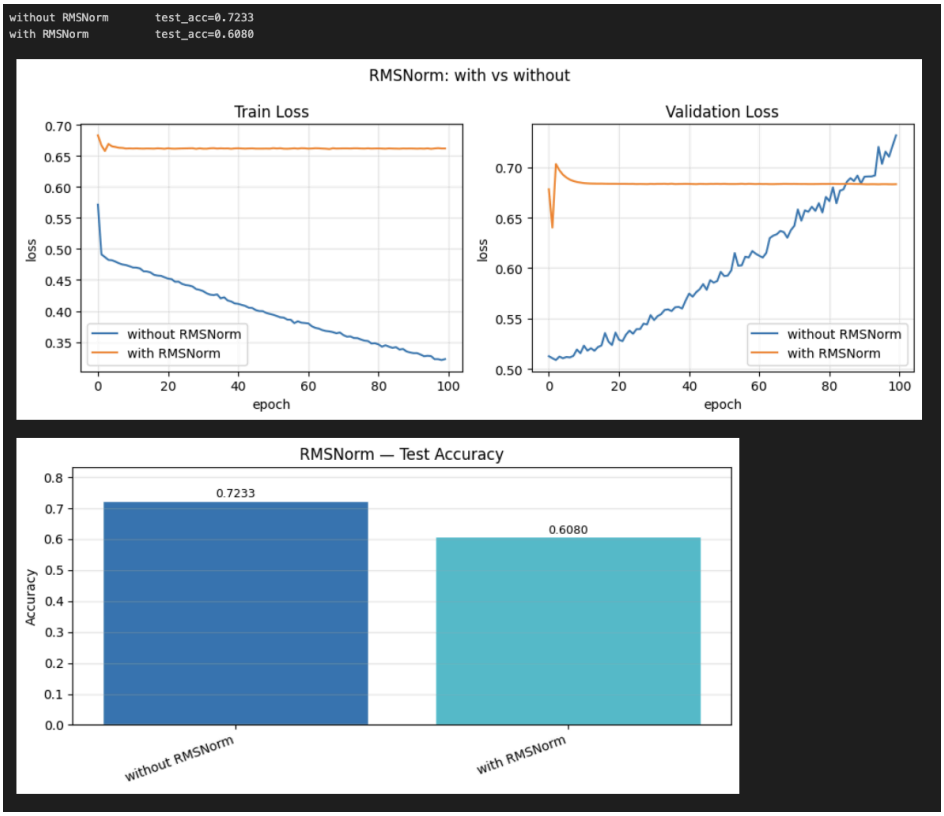
2.2.7 Pengaruh normalisasi RMSNorm

Arsitektur: [24, 64, 32, 1] dengan aktivasi ReLU dan Sigmoid. Optimizer Adam (lr=0.001), 100 epoch, batch size 64. Perbandingan satu model tanpa RMSNorm vs. satu model dengan RMSNorm (disisipkan setelah setiap Dense layer, sebelum aktivasi).

```
rms_configs = {'without RMSNorm': False, 'with RMSNorm': True}
rms_histories, rms_accs = {}, {}

for name, use_rms in rms_configs.items():
  model = FFNN(BASE_SIZES, ['relu', 'relu', 'sigmoid'], use_rmsnorm=use_rms)
  opt   = Adam(model.parameters(), learning_rate=LR)
  hist  = train(model, X_train, y_train, opt, bce_loss,
              epochs=EPOCHS, batch_size=BS, X_val=X_val, y_val=y_val, verbose=0)
  acc   = accuracy(model.predict(X_test), y_test)
  rms_histories[name], rms_accs[name] = hist, acc
  print(f'{name:20s}  test_acc={acc:.4f}')

compare_histories(list(rms_histories.values()), list(rms_histories.keys()), title='RMSNorm: with
vs without')
bar_comparison(list(rms_accs.keys()), list(rms_accs.values()), title='RMSNorm — Test
Accuracy')
```



Konfigurasi	Test Accuracy
Tanpa RMSNorm	72,33%
Dengan RMSNorm	60,8%

Analisis :
Pada eksperimen ini, penambahan RMSNorm justru menurunkan akurasi secara signifikan (dari 72.33% menjadi 60.80%). Beberapa faktor yang menjelaskan hasil ini:

- Dataset tabular dengan fitur yang sudah dinormalisasi: Data input telah melalui standardisasi (zero mean, unit variance) sebelum masuk ke model. Penambahan RMSNorm pada setiap hidden layer berarti re-normalisasi yang berulang-ulang, yang dapat menghilangkan informasi magnitude yang penting bagi klasifikasi.
- Jaringan yang dangkal: RMSNorm dirancang untuk jaringan yang sangat dalam (seperti Transformer) di mana gradients perlu distabilkan. Untuk jaringan dengan 2 hidden layer, manfaat stabilisasi gradien tidak sebanding dengan overhead komputasi dan potensi kehilangan informasi.
- Interaksi dengan aktivasi ReLU: RMSNorm me-rescale pre-activation values, yang dapat mengubah distribusi input ke ReLU secara drastis dan menyebabkan lebih banyak neuron "mati" (dying ReLU).

Dalam konteks ini, RMSNorm tidak memberikan manfaat dan bahkan merugikan. Namun perlu dicatat bahwa ini adalah hasil spesifik untuk arsitektur dan dataset yang digunakan.

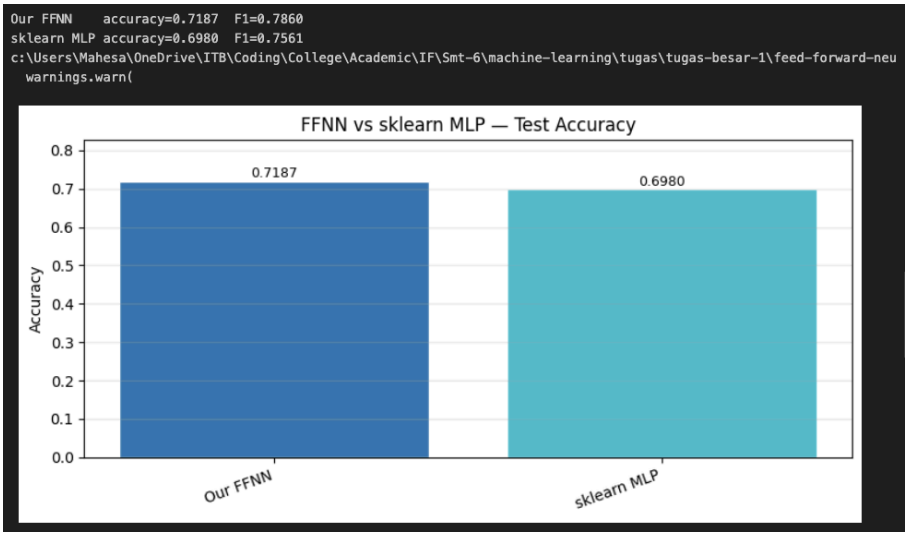
2.2.8 Perbandingan dengan library sklearn

Kedua model menggunakan arsitektur yang ekuivalen: dua hidden layer [64, 32], aktivasi ReLU, optimizer Adam (lr=0.001), 100 epoch, batch size 64, data training/test yang sama.

```
# our FFNN
model_ours = FFNN(BASE_SIZES, ['relu', 'relu', 'sigmoid'])
opt_ours   = Adam(model_ours.parameters(), learning_rate=LR)
train(model_ours, X_train, y_train, opt_ours, bce_loss,
    epochs=EPOCHS, batch_size=BS, verbose=0)
ours_pred = (model_ours.predict(X_test) >= 0.5).astype(int).flatten()
ours_acc  = accuracy_score(y_test.flatten(), ours_pred)
ours_f1   = f1_score(y_test.flatten(), ours_pred)

# sklearn MLP
mlp = MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu',
                    solver='adam', learning_rate_init=LR,
                    max_iter=EPOCHS, batch_size=BS, random_state=42)
mlp.fit(X_train, y_train.ravel())
sk_pred = mlp.predict(X_test)
sk_acc  = accuracy_score(y_test.flatten(), sk_pred)
sk_f1   = f1_score(y_test.flatten(), sk_pred)

print(f'Our FFNN    accuracy={ours_acc:.4f}  F1={ours_f1:.4f}')
print(f'sklearn MLP accuracy={sk_acc:.4f}  F1={sk_f1:.4f}')
bar_comparison(['Our FFNN', 'sklearn MLP'], [ours_acc, sk_acc],
            title='FFNN vs sklearn MLP — Test Accuracy')
```



Model	Test Accuracy	F1-Score
FFNN Implementasi Kami	71.87%	78.60%
sklearn MLPClassifier	69.80%	75.61%

Catatan: sklearn MLPClassifier mengeluarkan ConvergenceWarning model belum konvergen setelah 100 iterasi.

Analisis
FFNN implementasi kami mengungguli sklearn MLPClassifier pada kedua metrik (accuracy dan F1-score). Hal ini menunjukkan bahwa implementasi from scratch telah berhasil dan benar secara matematis.

Keunggulan pada F1-score (78.60% vs 75.61%) cukup signifikan dan mengindikasikan bahwa model kami lebih baik dalam menyeimbangkan precision dan recall untuk dataset yang sedikit imbalanced ini. ConvergenceWarning pada sklearn menunjukkan bahwa model sklearn membutuhkan lebih dari 100 iterasi untuk konvergen, sedangkan model kami (71.87% accuracy) sudah memberikan prediksi yang lebih stabil meski menggunakan hyperparameter yang identik.

Perbedaan performa ini juga dapat dipengaruhi oleh detail implementasi internal: sklearn menggunakan optimasi internal tertentu (misal, early stopping berbasis tolerance) yang berbeda dari implementasi kami.

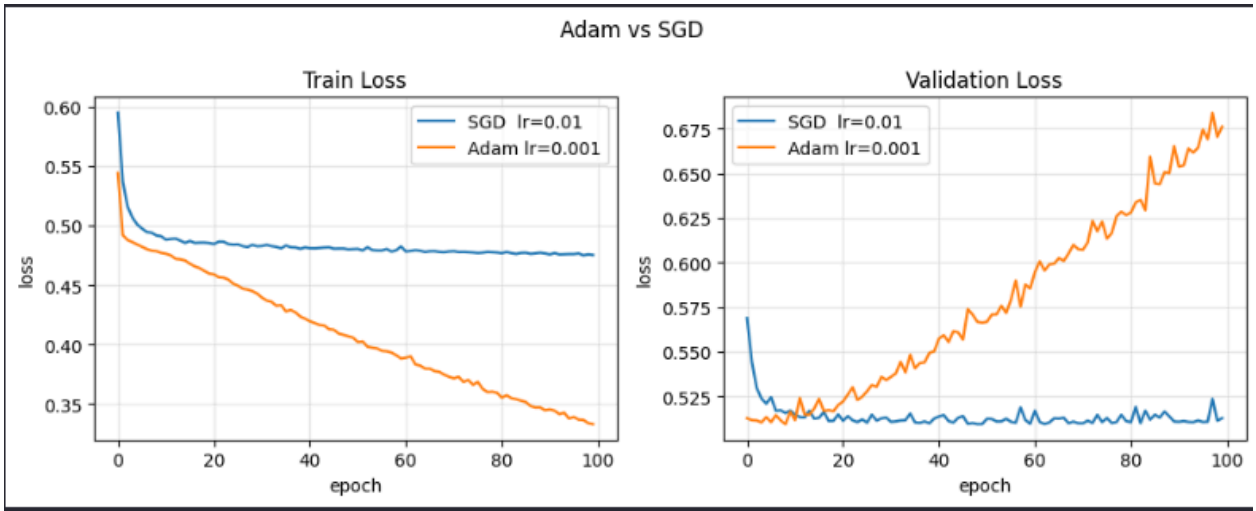
2.2.9 Perbandingan Konvergensi Adam vs SGD

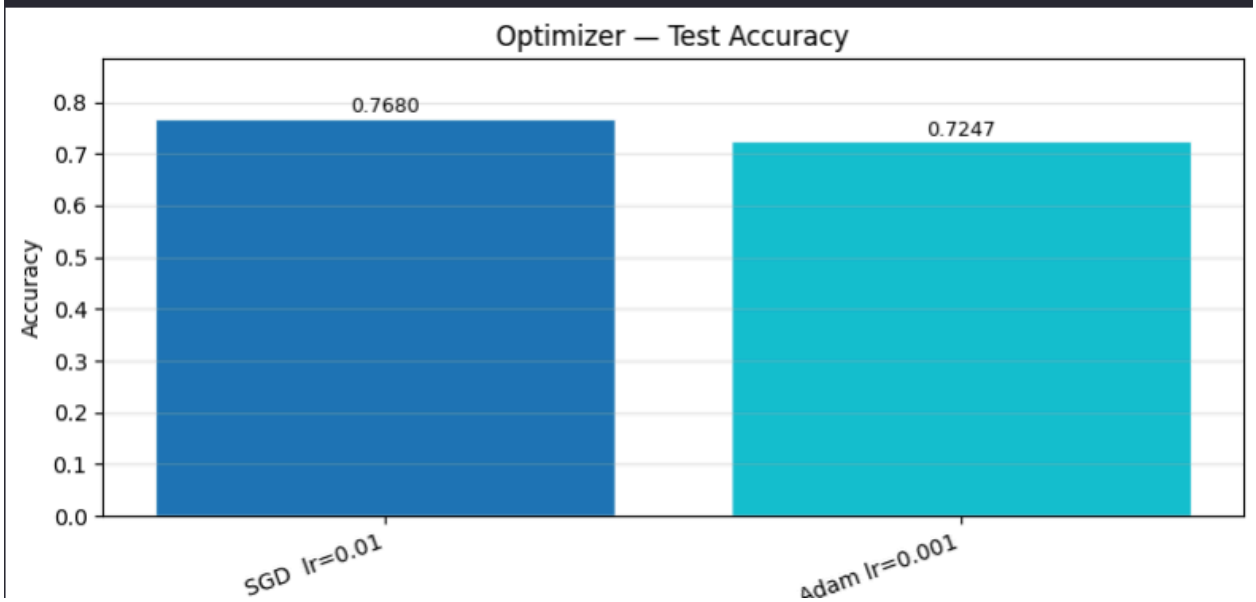
Eksperimen ini membandingkan konvergensi dua optimizer, SGD (lr=0.01) dan Adam (lr=0.001), pada arsitektur yang identik, untuk mengamati perbedaan karakteristik dan kecepatan belajar keduanya.

```
optim_configs = {
    'SGD  lr=0.01':  lambda p: SGD(p,  learning_rate=0.01),
    'Adam lr=0.001': lambda p: Adam(p, learning_rate=0.001),
}
optim_histories, optim_accs = {}, {}

for name, opt_fn in optim_configs.items():
    model = FFNN(BASE_SIZES, ['relu', 'relu', 'sigmoid'])
    hist  = train(model, X_train, y_train,
opt_fn(model.parameters()), bce_loss,
                  epochs=EPOCHS, batch_size=BS, X_val=X_val,
y_val=y_val, verbose=0)
    acc   = accuracy(model.predict(X_test), y_test)
    optim_histories[name], optim_accs[name] = hist, acc
    print(f'{name}  test_acc={acc:.4f}')

compare_histories(list(optim_histories.values()),
list(optim_histories.keys()), title='Adam vs SGD')
bar_comparison(list(optim_accs.keys()),
list(optim_accs.values()), title='Optimizer — Test Accuracy')
```





Optimizer	Learning Rate	Test Accuracy
SGD	0.01	76.80%
Adam	0.001	72.47%

Analisis

Pada eksperimen ini, SGD dengan lr=0.01 menghasilkan akurasi test yang lebih tinggi (76.80%) dibandingkan Adam dengan lr=0.001 (72.47%). Hasil ini tampak berlawanan dengan ekspektasi umum, namun dapat dijelaskan oleh beberapa faktor:

- Adam lebih agresif pada awal training: Adam menyesuaikan learning rate secara adaptif per parameter, yang menyebabkan konvergensi training loss lebih cepat.

Namun pada dataset tabular yang relatif kecil dan sederhana ini, kecepatan tersebut berujung pada overfitting yang lebih parah, terbukti dari val_loss Adam yang melonjak jauh lebih tinggi dibandingkan SGD seiring berjalannya epoch.

- SGD lebih stabil secara implisit: SGD dengan learning rate tetap menghasilkan pembaruan bobot yang lebih konsisten dan tidak terlalu agresif, sehingga model lebih mampu mempertahankan generalisasi pada data validasi dan test.
- Perbedaan learning rate berperan: Kedua optimizer menggunakan learning rate berbeda (SGD: 0.01 vs Adam: 0.001). SGD biasanya membutuhkan lr lebih besar karena tidak memiliki mekanisme adaptif. Meski begitu, perbandingan ini tetap valid karena mencerminkan konfigurasi tipikal yang digunakan di praktik.
- Konteks berbeda dengan eksperimen sebelumnya: Pada eksperimen regularisasi, width, dan depth, Adam (lr=0.001) memberikan hasil yang baik karena dikombinasikan dengan regularisasi atau arsitektur yang berbeda. Tanpa regularisasi dan pada arsitektur [24, 64, 32, 1], SGD terbukti lebih stabil untuk dataset ini.

Kesimpulannya, pemilihan optimizer yang optimal bergantung pada konteks dataset, arsitektur, dan apakah regularisasi diterapkan. Adam tidak selalu lebih baik dari SGD, khususnya pada masalah yang relatif sederhana di mana overfitting adalah tantangan utama.

BAB III : KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan implementasi dan serangkaian eksperimen yang telah dilakukan terhadap Feedforward Neural Network from scratch pada dataset Global Student Placement & Salary, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Implementasi FFNN from scratch berhasil dilakukan dengan benar
   Validasi melalui gradient check numerik membuktikan bahwa implementasi automatic differentiation (autograd) menghasilkan gradien yang akurat dengan error maksimum 3.22×10^{-11}, jauh di bawah toleransi 10^{-5}.
2. Pengaruh depth dan width
   Pada dataset ini, model yang lebih kecil (shallow dan narrow) memberikan generalisasi yang lebih baik. Model deep [64,32,16,8] menghasilkan akurasi 70.00% dibandingkan shallow [64] yang mencapai 75.40%, menunjukkan bahwa kapasitas model yang berlebihan menyebabkan overfitting pada dataset dengan ukuran sedang.
3. Pengaruh fungsi aktivasi: Sigmoid pada hidden layer memberikan akurasi terbaik (76.93%) pada dataset tabular ini, diikuti Linear (76.67%) dan ELU (75.53%). ReLU kurang optimal (70.47%) kemungkinan karena dying ReLU problem. Pemilihan fungsi aktivasi sangat bergantung pada karakteristik dataset.
4. Pengaruh learning rate
   Learning rate 0.01 dengan SGD memberikan performa terbaik (77.00%), sedangkan learning rate terlalu besar (0.1) menyebabkan ketidakstabilan training (72.67%) dan learning rate terlalu kecil (0.001) belum konvergen dalam 100 epoch (75.13%). Trade-off antara kecepatan konvergensi dan stabilitas harus dipertimbangkan saat memilih learning rate.
5. Pengaruh inisialisasi bobot
   Zero initialization adalah pilihan yang buruk karena symmetry problem (60.73%). Uniform initialization memberikan akurasi test terbaik (74.33%), sementara Xavier/He initialization memberikan training loss lebih rendah tetapi rentan terhadap overfitting pada dataset ini.
6. Pengaruh regularisasi
   Regularisasi terbukti penting dalam mengatasi overfitting. L1 memberikan peningkatan terbesar (76.67% vs 72.00% tanpa regularisasi) karena sifat sparsifikasi bobotnya cocok untuk dataset dengan banyak fitur kategorik. L2 juga efektif (75.13%).
7. RMSNorm
   Tidak memberikan manfaat pada jaringan dangkal dengan dataset tabular yang sudah dinormalisasi (60.80% vs 72.33% tanpa RMSNorm). RMSNorm lebih cocok digunakan pada arsitektur sangat dalam seperti Transformer.
8. Perbandingan dengan sklearn
   FFNN implementasi kami mengungguli sklearn MLPClassifier (71.87% vs 69.80% accuracy, 78.60% vs 75.61% F1-score), memvalidasi kebenaran dan efektivitas implementasi.

3.2 Saran

Dari keseluruhan proses implementasi dan pengujian arsitektur from scratch, kami menemukan berbagai wawasan menarik terkait performa hyperparameter. Untuk menyempurnakan performa, generalisasi, dan efisiensi arsitektur di masa mendatang, kami merekomendasikan beberapa saran berikut:

1. Tuning regularisasi lebih lanjut
   Mencoba nilai λ yang lebih kecil (10^{-4}) atau kombinasi L1 dan L2 (Elastic Net) untuk mencari titik optimal antara bias dan variance.
2. Early stopping
   Implementasi mekanisme early stopping berbasis validation loss untuk secara otomatis menghentikan pelatihan saat model mulai overfitting, menggantikan jumlah epoch tetap.
3. Hyperparameter search
   Menggunakan grid search atau random search untuk menemukan kombinasi optimal dari learning rate, regularisasi, dan arsitektur secara sistematis.
4. Feature engineering
   Eksplorasi tambahan pada preprocessing, seperti polynomial features atau interaction terms, untuk meningkatkan kapasitas representasi tanpa harus memperdalam jaringan.
5. Eksperimen lebih lanjut dengan Adam
   Adam menunjukkan konvergensi yang lebih cepat dan stabil dibandingkan SGD. Kombinasi Adam dengan regularisasi L1 dapat dijelajahi untuk mendapatkan performa optimal.

BAB IV : PEMBAGIAN TUGAS TIAP ANGGOTA KELOMPOK

Nama	NIM	Pembagian Tugas
Mahesa Fadhillah Andre	13523140	- Mengembangkan mesin Automatic Differentiation (Autodiff) pada folder engine. - Membangun struktur dasar jaringan, layer Dense, fungsi aktivasi, dan RMSNorm pada folder neuron. - Menyusun kelas utama model pada folder models. - Merancang alur pelatihan model pada file train.py.
Jonathan Kenan Budianto	13523139	- Mengimplementasikan pengoptimal Adam dan SGD pada folder optim. - Mengembangkan fungsi inisialisasi bobot (Xavier, He) dan fungsi Loss (BCE, CCE, MSE) pada folder optim. - Membuat modul pra pemrosesan data (standardisasi, one-hot) dan visualisasi grafik pada folder utils.
Sebastian Enrico Nathanael	13523134	- Mengintegrasikan seluruh modul ke dalam notebook interaktif pada file main.ipynb. - Melakukan eksekusi eksperimen variasi hyperparameter (depth, width, aktivasi, learning rate). - Melakukan analisis data, ekstraksi metrik akurasi, dan penyusunan laporan hasil pengujian.

BAB V : REFERENSI

1. Karpathy, A. The spelled-out intro to neural networks and backpropagation: building micrograd. https://youtu.be/VMj-3S1tku0
2. Osajima, J. Forward Propagation. https://www.jasonosajima.com/forwardprop
3. Osajima, J. Backpropagation. https://www.jasonosajima.com/backprop
4. NumPy Documentation v2.2. https://numpy.org/doc/2.2/
5. scikit-learn MLPClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
6. Greenplace, E. The Softmax function and its derivative. https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/
7. Orr, D. Automatic Differentiation Tutorial. https://douglasorr.github.io/2021-11-autodiff/article.html
8. Stanford University. (n.d.). *CS231n: Convolutional Neural Networks for Visual Recognition - Neural Networks Notes*. https://cs231n.github.io/neural-networks-1/
9. Grosse, R. Automatic Differentiation Tutorial. https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf
10. Kingma, D., Ba, J. Adam: A Method for Stochastic Optimization. https://arxiv.org/abs/1412.6980
11. Training and Validation Loss in Deep Learning. https://www.geeksforgeeks.org/training-and-validation-loss-in-deep-learning/
12. L2 Regularization. https://builtin.com/data-science/l2-regularization
13. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. http://www.deeplearningbook.org
14. Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*. https://arxiv.org/abs/1412.6980
15. Zhang, B., & Sennrich, R. (2019). Root Mean Square Layer Normalization. *Advances in Neural Information Processing Systems*, 32. https://arxiv.org/abs/1910.07467
16. The pandas development team. (2024). *pandas-dev/pandas: Pandas*. Zenodo. https://pandas.pydata.org/
17. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95.

BAB VI : LAMPIRAN

6.1 Lampiran Surat Pernyataan Penggunaan AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*

Nama/NIM	: Sebastian Enrico Nathanael/13523134
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah	: IF3270
Nama Mata Kuliah	: Pembelajaran Mesin
Judul Tugas/Proposal	: Tugas Besar 1 IF3270 Pembelajaran Mesin Feedforward Neural Network

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	Tingkat 1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	Tingkat 2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	Tingkat 2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	Tingkat 1
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Surabaya, 18 Maret 2026

Sebastian Enrico Nathanael

13523134

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*

Nama/NIM	: Jonathan Kenan Budianto/13523139
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah	: IF3270
Nama Mata Kuliah	: Pembelajaran Mesin
Judul Tugas/Proposal	: Tugas Besar 1 IF3270 Pembelajaran Mesin Feedforward Neural Network

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	Tingkat 1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	Tingkat 2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	Tingkat 2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	Tingkat 1
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Jakarta, 18 Maret 2026

Jonathan Kenan Budianto

13523139

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*

Nama/NIM	: Mahesa Fadhillah Andre/13523140
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah	: IF3270
Nama Mata Kuliah	: Pembelajaran Mesin
Judul Tugas/Proposal	: Tugas Besar 1 IF3270 Pembelajaran Mesin Feedforward Neural Network

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

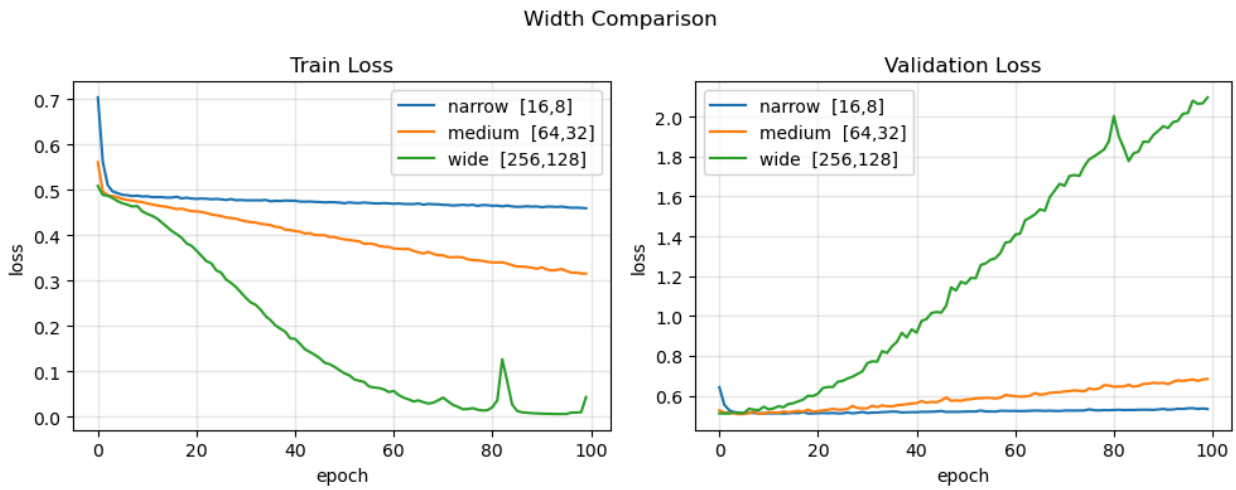
Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	Tingkat 1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	Tingkat 2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	Tingkat 2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	Tingkat 1
Lainnya, Jelaskan:		

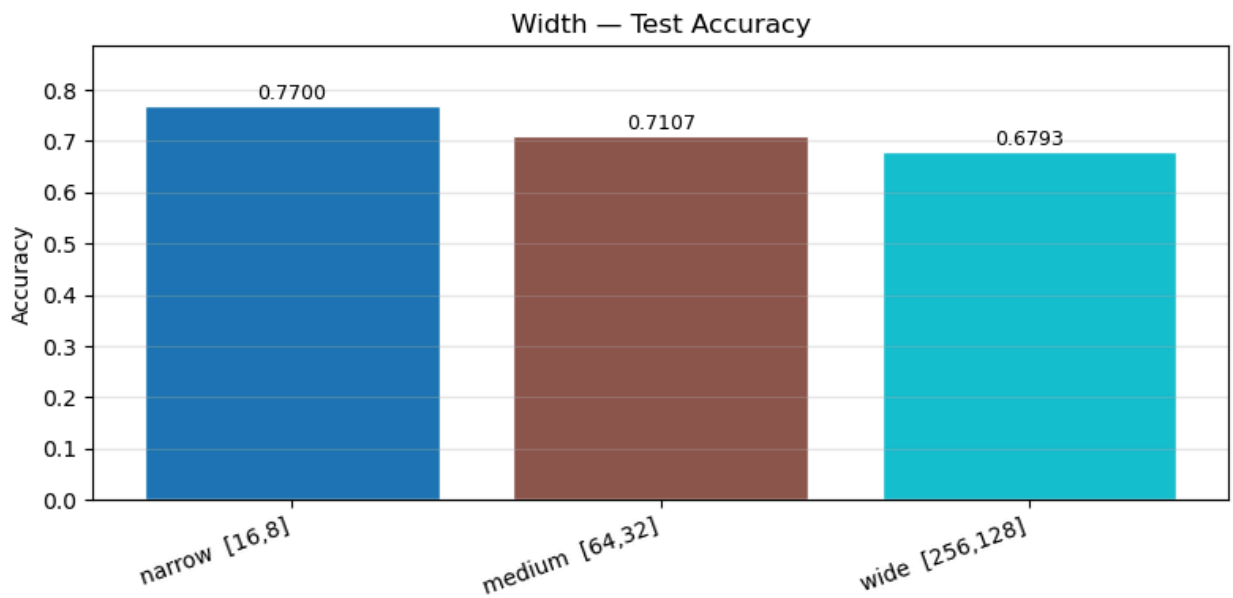
Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

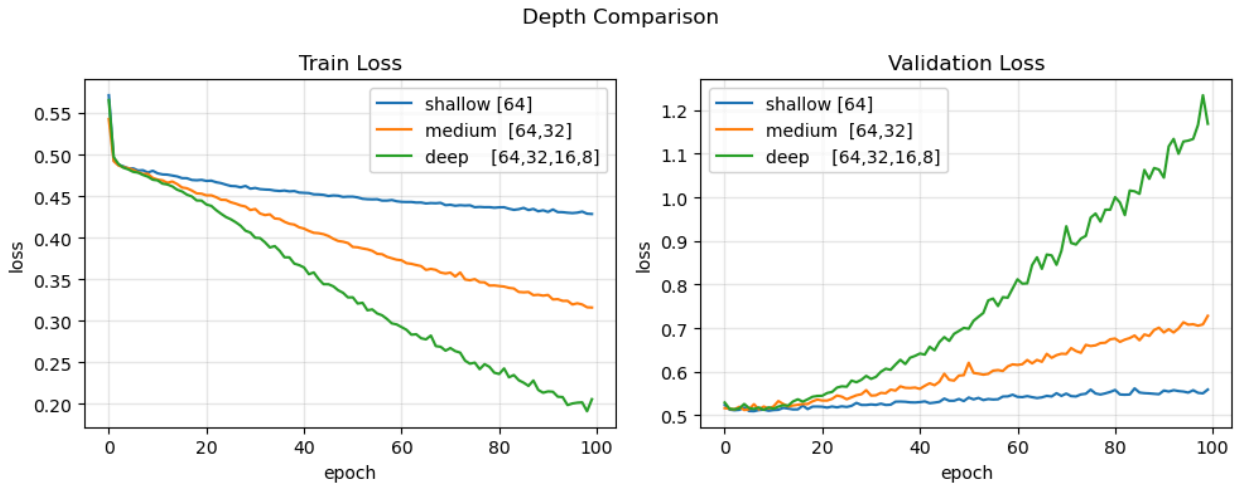
Jakarta, 18 Maret 2026

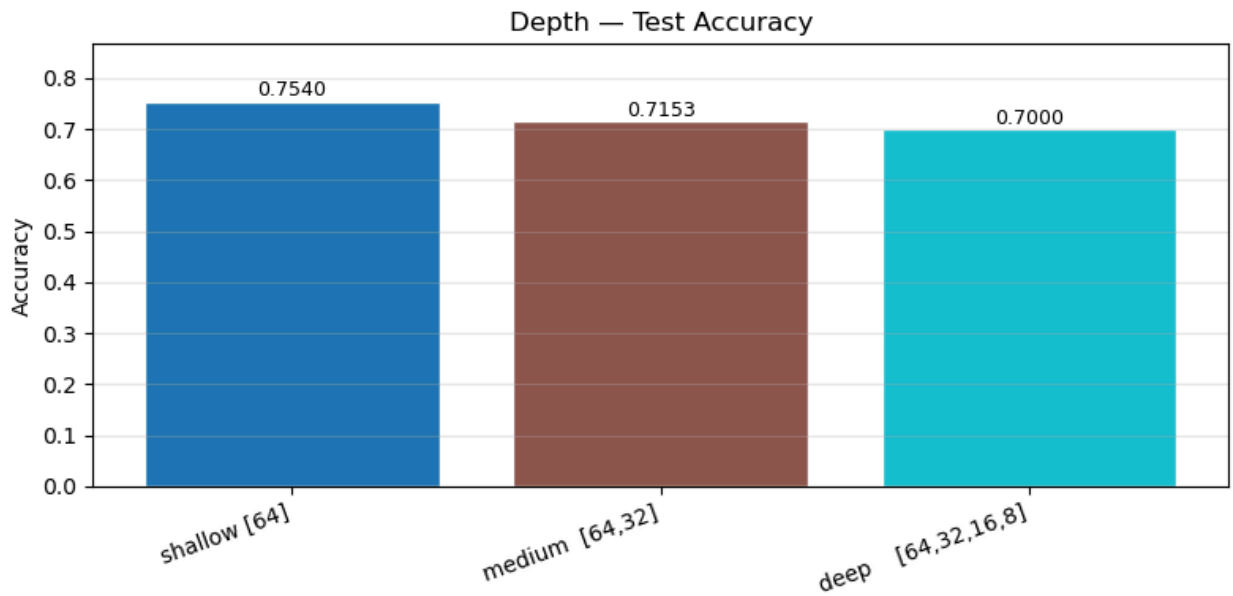
Mahesa Fadhillah Andre

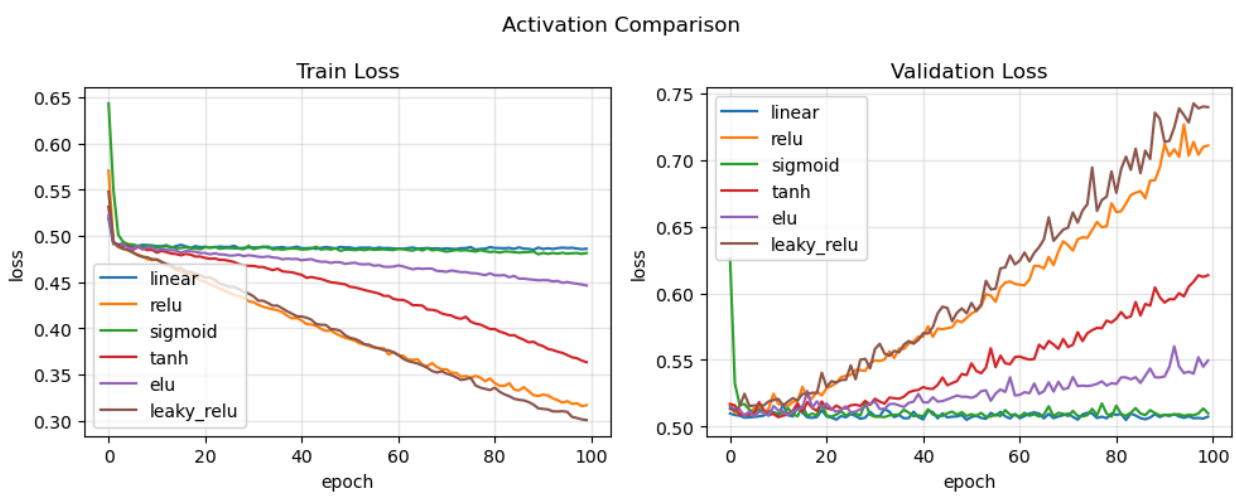
6.2 Lampiran Gambar Hasil Notebook

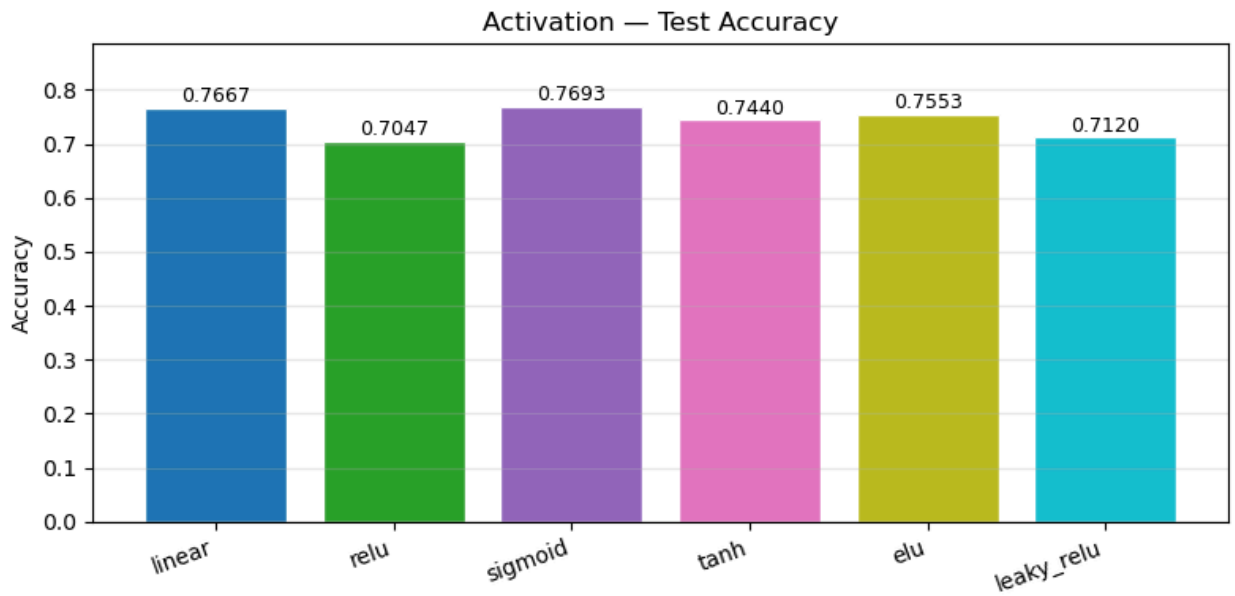


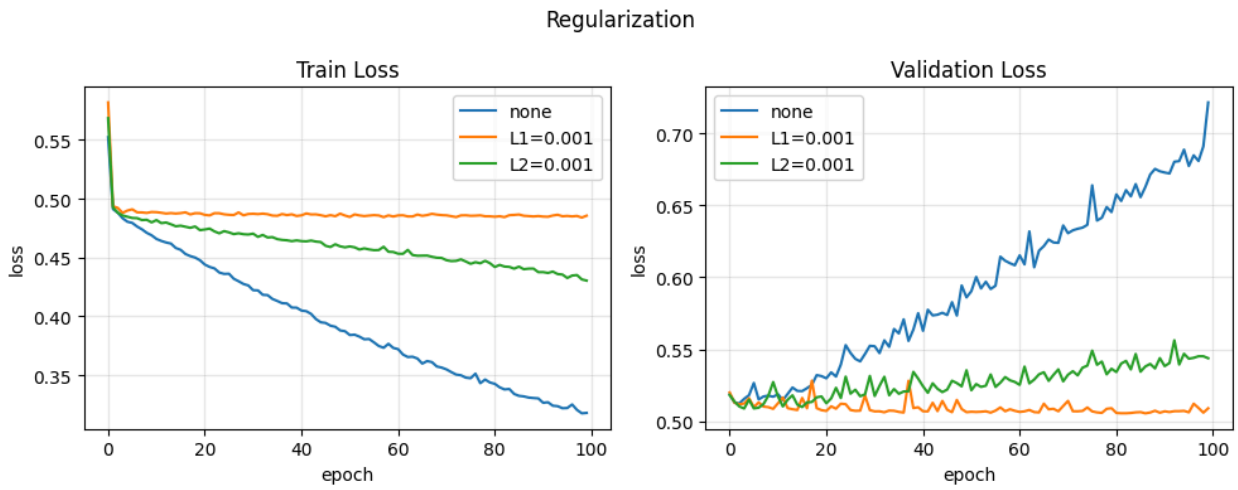


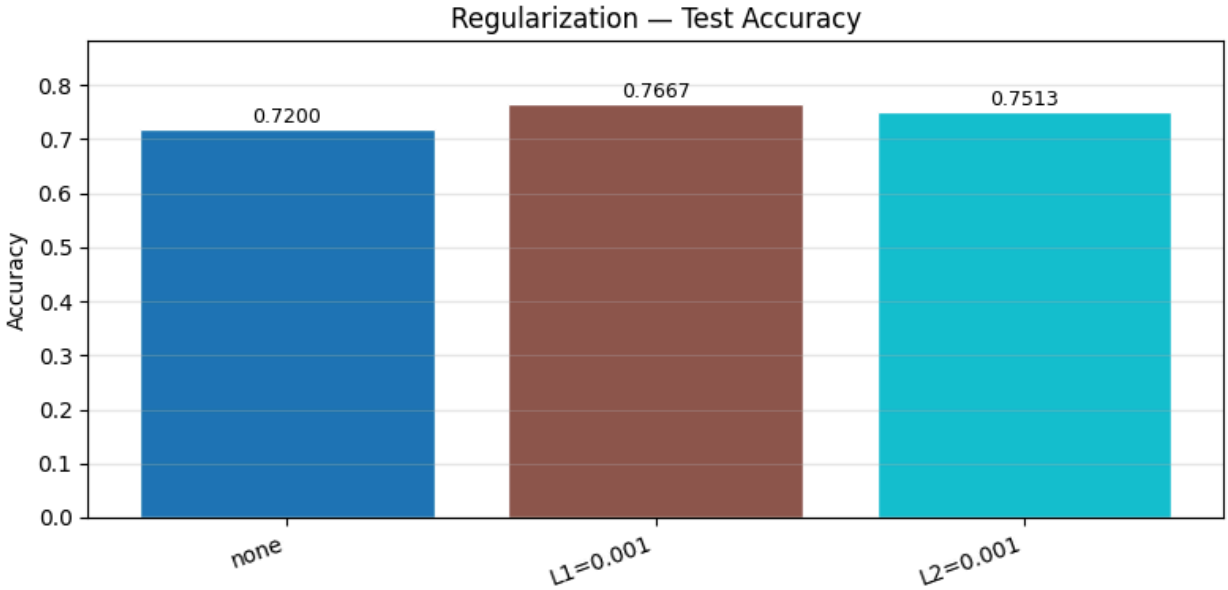

Depth Comparison
Train Loss
Validation Loss
shallow [64]
medium [64,32]
deep [64,32,16,8]
loss
epoch

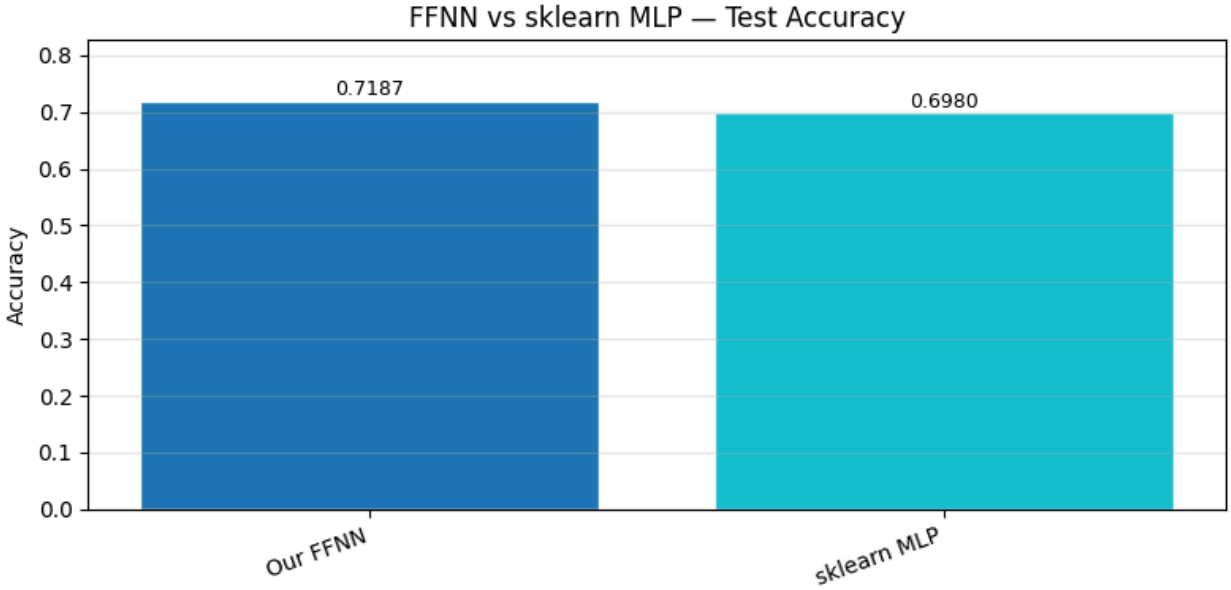

Depth — Test Accuracy
Accuracy
0.7540
0.7153
0.7000
shallow [64]
medium [64,32]
deep [64,32,16,8]

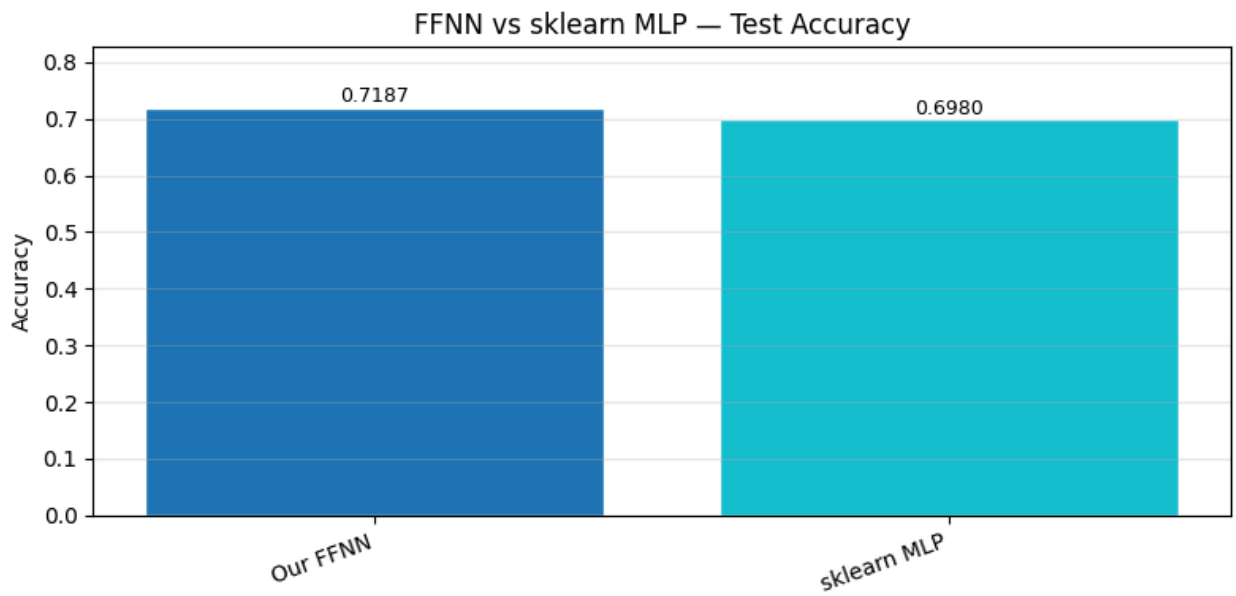

Activation Comparison
Train Loss
Validation Loss
linear
relu
sigmoid
tanh
elu
leaky_relu
loss
epoch

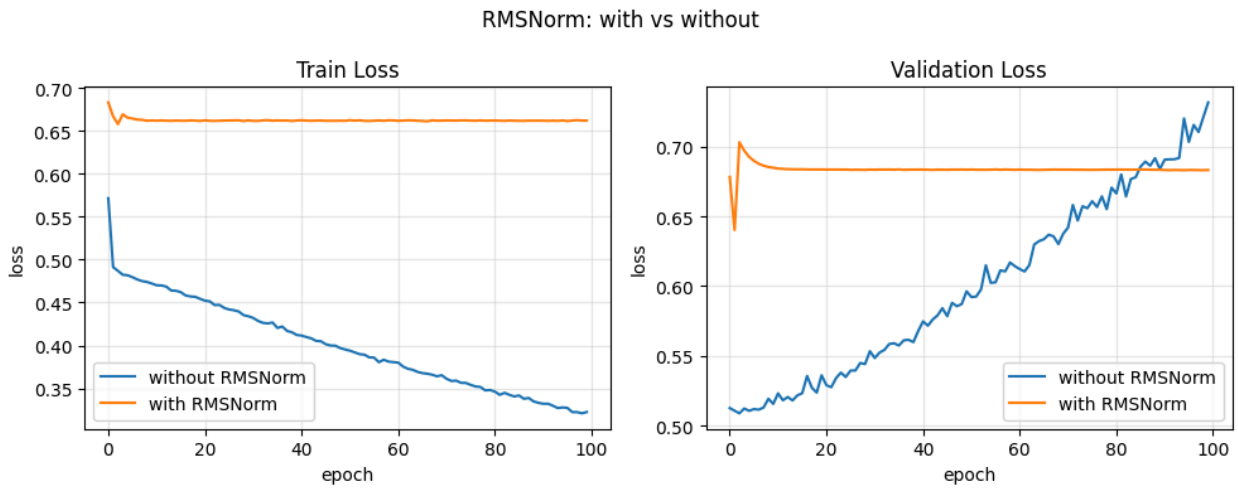

Activation — Test Accuracy
Accuracy
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.7667
0.7047
0.7693
0.7440
0.7553
0.7120
linear
relu
sigmoid
tanh
elu
leaky_relu

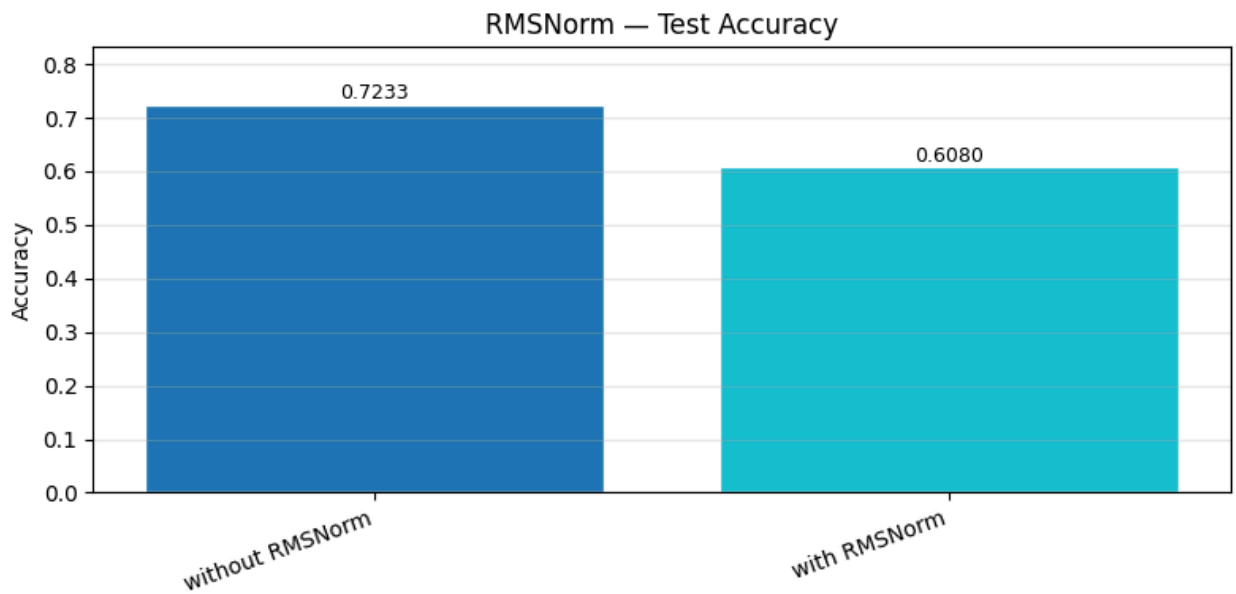

Regularization
Train Loss
loss
epoch
none
L1=0.001
L2=0.001
Validation Loss
loss
epoch
none
L1=0.001
L2=0.001


Regularization — Test Accuracy
Accuracy
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.7200
0.7667
0.7513
none
L1=0.001
L2=0.001

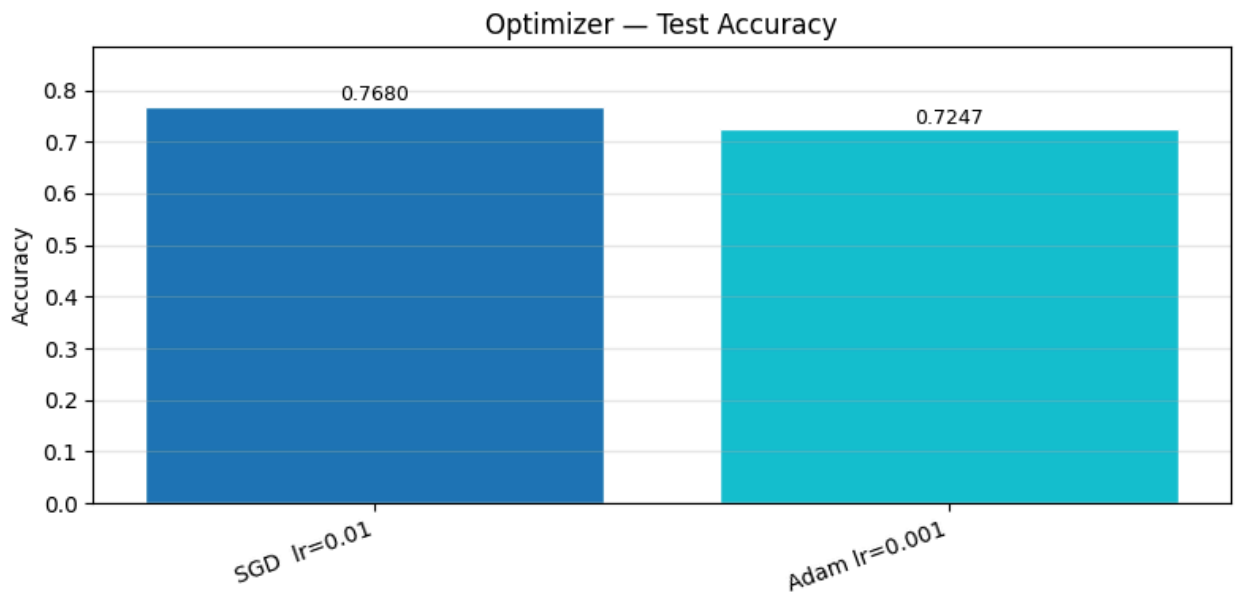

FFNN vs sklearn MLP — Test Accuracy
Accuracy
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.7187
0.6980
Our FFNN
sklearn MLP

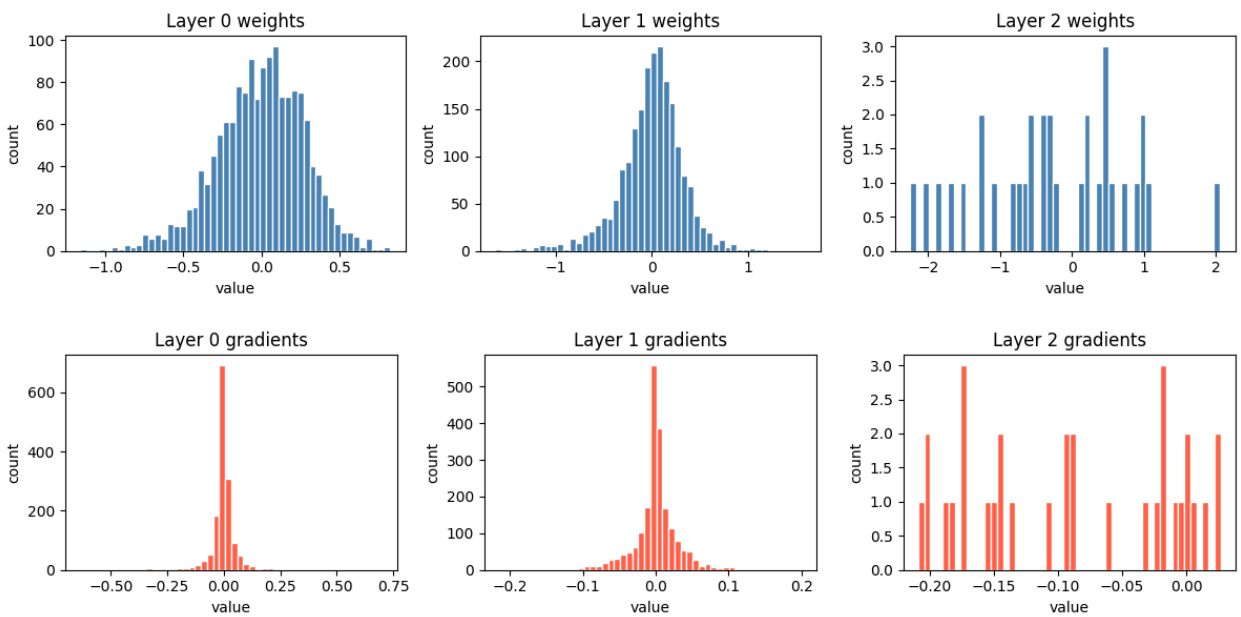

FFNN vs sklearn MLP — Test Accuracy
Accuracy
0.7187
0.6980
Our FFNN
sklearn MLP


RMSNorm: with vs without
Train Loss
Validation Loss
loss
epoch
without RMSNorm
with RMSNorm


RMSNorm — Test Accuracy
Accuracy
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.7233
0.6080
without RMSNorm
with RMSNorm

Adam vs SGD
Train Loss
Validation Loss
SGD lr=0.01
Adam lr=0.001
loss
epoch
0
20
40
60
80
100
0.35
0.40
0.45
0.50
0.55
0.60
0.525
0.550
0.575
0.600
0.625
0.650
0.675


Optimizer — Test Accuracy
Accuracy
0.7680
0.7247
SGD lr=0.01
Adam lr=0.001


Layer 0 weights
Layer 1 weights
Layer 2 weights
Layer 0 gradients
Layer 1 gradients
Layer 2 gradients
count
value